BookForMe: An Agentic AI-Powered Centralized Booking Platform

Kaavish Report

presented to the academic faculty

by

Muhammad Taqi	mt08073
Muhammad Ahmad Humayun Hanif	mh08072
Jazib Waqas	jw08048
Taha Hunaid Ali	ta08451

In partial fulfillment of the requirements for

Bachelor of Science

Computer Science

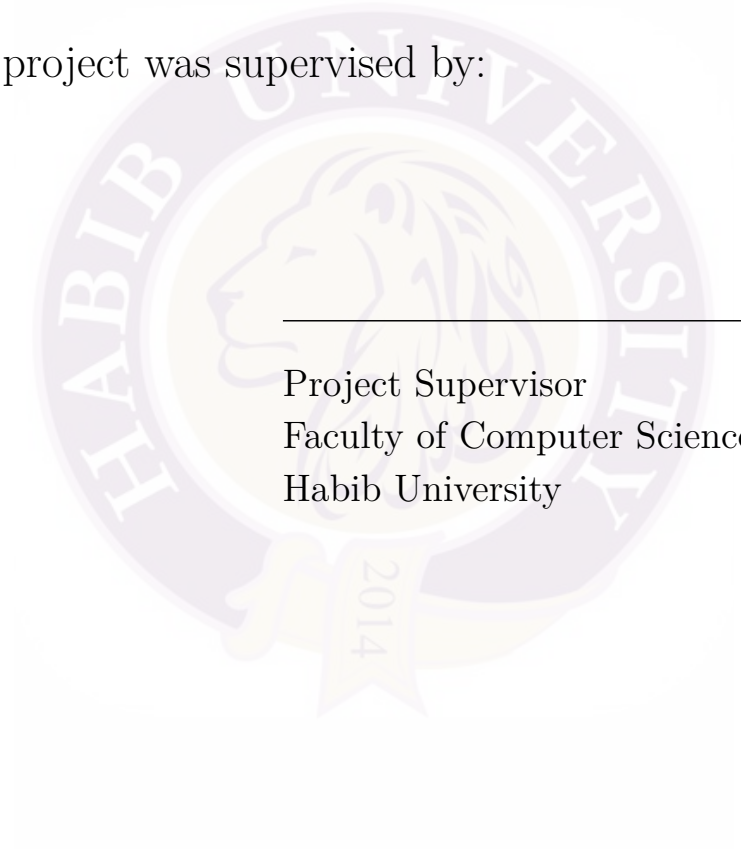
Dhanani School of Science and Engineering

Habib University

Spring 2026

BookForMe: An Agentic AI-Powered Centralized Booking Platform

This Kaavish project was supervised by:



Project Supervisor
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____

Acknowledgements

We want to thank the CS faculty at Habib University for their guidance throughout the Kaavish programme, particularly during milestone reviews where their feedback sharpened our system design and evaluation methodology.

We are grateful to the futsal and padel court operators in Karachi who generously shared their WhatsApp booking conversations and allowed us to observe real scheduling workflows. Without their cooperation, our bilingual dataset would not exist.

Finally, we acknowledge the open-source communities behind HuggingFace Transformers, LangGraph, FastAPI, and React Native, whose work made this project technically feasible within one academic year.

Abstract

Booking recreational and informal services in Pakistan (futsal courts, salons, gaming zones, and beach houses) remains fragmented and inefficient. Customers must individually contact vendors over WhatsApp, wait for manual availability checks, and confirm reservations through unstructured conversation. This process is slow, prone to double-bookings, and poorly suited to users who communicate in Roman Urdu.

BookForMe is a centralized, agentic AI booking platform that addresses these problems. It provides a React Native mobile application for browsing and booking services, and an AI Receptionist that autonomously manages booking requests on the vendor's WhatsApp channel. Core contributions are: (1) a custom bilingual dataset of 657+ annotated utterances in English and Roman Urdu constructed through real vendor conversations and LLM-driven augmentation; (2) a comparative evaluation of six transformer architectures for intent classification, with DistilBERT achieving the best performance of 90.5% accuracy and 91.7% macro-F1; (3) a LangGraph-based cyclic state machine implementing the full booking pipeline with Optimistic Concurrency Control (OCC) in Firestore and OCR-based payment verification; and (4) a social layer with Elo-based matchmaking, community forums, and leaderboards.

A user survey ($n > 50$) found that 72% of respondents rely on WhatsApp or phone calls for bookings and 81% reported frustration with multi-vendor coordination, validating the social relevance of this work. The platform is live at https://jhat-to9p.onrender.com and open-sourced at https://github.com/JazibWaqas/JHAT.

Keywords: Agentic AI, Task-Oriented Dialogue, Bilingual NLU, Roman Urdu, LangGraph, Optimistic Concurrency Control, WhatsApp Business API, DistilBERT.

Contents

Acknowledgements **2**

Abstract **3**

1 Introduction **12**
1.1 Problem Statement 12
1.2 Proposed Solution 14
1.3 Intended Users 15
1.4 Project Timeline and Deliverables 16
1.5 Key Challenges 17

2 Literature Review **19**
2.1 From Chatbots to Autonomous Agents 19
2.1.1 Defining the Agentic Architecture 19
2.1.2 Cognitive Architectures: ReAct, PRACT, and Reflexion . . . 20
2.1.3 Orchestration Frameworks: LangChain and LangGraph 20
2.1.4 Multi-Agent Decomposition 21
2.2 Task-Oriented Dialogue and State Tracking 21
2.2.1 Why Separate NLU and NLG 22
2.3 Bilingual and Code-Switched NLU 22
2.3.1 Roman Urdu Challenges 22
2.3.2 Cross-Lingual Transfer and Adapters 23
2.3.3 Pretrained Models for Classification 23
2.3.4 Data Augmentation for Low-Resource Settings 24
2.4 Retrieval, Planning, and Tool Use 24
2.5 Payment Verification via Computer Vision 25
2.6 Distributed Booking and Concurrency Safety 25
2.7 Social Ranking: Elo Rating Systems 26

2.8 Novelty and Gap Analysis 26

3 Software Requirement Specification (SRS) 28

3.1 Introduction 28
3.1.1 Purpose 28
3.1.2 Scope 28
3.2 Functional Requirements 29
3.2.1 Module: User (Booking & Discovery) 29
3.2.2 Module: User (AI Chat & Discovery) 30
3.2.3 Module: User (Social & Community) 30
3.2.4 Module: User (Matchmaking & Ranking) 30
3.2.5 Module: Vendor 31
3.2.6 Module: AI Agent (WhatsApp Receptionist) 31
3.2.7 Module: Admin 32
3.2.8 Use Case Diagrams 32
3.3 Non-Functional Requirements 33
3.3.1 Performance 33
3.3.2 Security 33
3.3.3 Reliability 34
3.3.4 Scalability 34
3.3.5 Usability 34
3.4 System Block Diagram 35
3.4.1 Component descriptions 35
3.5 System Screenshots 36
3.5.1 Authentication 37
3.5.2 Customer application 38
3.5.3 Social hub 40
3.5.4 Vendor dashboard 42
3.5.5 AI receptionist (WhatsApp) 43
3.5.6 Admin control panel 44

4 Software Design Specification (SDS) 45
4.1 System Architecture . . . 45
4.1.1 Architectural Design . . . 45
4.1.2 System Architecture Diagram . . . 45
4.2 Data Model (Data Design) . . . 46
4.2.1 Database Strategy . . . 46
4.2.2 Entity Descriptions . . . 46
4.2.3 NoSQL Schema Diagram . . . 47
4.3 Agentic Booking Process Pipeline . . . 48
4.3.1 Pipeline Logic . . . 49
4.4 Technical Details & Workflows . . . 50
4.4.1 Key Algorithms . . . 50
4.4.2 Workflow Visualizations . . . 51
4.5 Security Implementation . . . 55

5 Development 56
5.1 Overview . . . 56
5.2 Database Design Decisions . . . 56
5.2.1 Slot as the Canonical Booking Record . . . 56
5.2.2 Composite Document IDs for Collision Prevention . . . 57
5.2.3 Two-Domain Schema . . . 57
5.2.4 Batch Reads to Eliminate N+1 Queries . . . 57
5.3 AI Receptionist Development . . . 58
5.3.1 Two-Model NLU and NLG Architecture . . . 58
5.3.2 LangGraph: Why a State Graph Over a Simple Pipeline . . . 58
5.3.3 Entity Validation: From Regex to Pydantic . . . 59
5.3.4 Guardrails Node . . . 59
5.4 Mobile Application Development . . . 60
5.4.1 Expo Go Constraints and Library Trade-offs . . . 60
5.4.2 Caching Strategy . . . 60
5.4.3 Role-Based Routing in a Single App . . . 60

5.4.4 Smart Filter (In-App AI Chat) . . . 61
5.4.5 Matchmaking and Elo Ranking . . . 61
5.5 Vendor Dashboard and Admin Panel . . . 61
5.6 Payment Verification via OCR . . . 62

6 Methodology, Experiments and Results 63
6.1 Dataset Construction . . . 63
6.1.1 Data Collection . . . 63
6.1.2 Dataset Statistics . . . 64
6.1.3 Annotation Schema . . . 64
6.2 NLU Model Training and Selection . . . 65
6.2.1 Training Configuration . . . 65
6.2.2 Model Comparison . . . 65
6.2.3 Selected Model: DistilBERT . . . 66
6.2.4 Per-Class Analysis (DistilBERT) . . . 67
6.3 Agent Evaluation . . . 68
6.3.1 Test Case Methodology . . . 68
6.3.2 Test Case Results . . . 69
6.4 Discussion . . . 70
6.4.1 Key Findings . . . 70
6.4.2 Limitations . . . 70

7 Conclusion and Future Work 72
7.1 Summary . . . 72
7.2 Future Work . . . 72

8 Reflection 74
8.1 Individual Reflections . . . 74
8.1.1 Muhammad Ahmad Humayun Hanif . . . 74
8.1.2 Jazib Waqas . . . 74
8.1.3 Taha Hunaid Ali . . . 75
8.1.4 Muhammad Taqi . . . 75

8.2 Team Reflection 76

References **77**

List of Figures

1.1 High-level overview of the BookForMe platform. 14

3.1 System block diagram for the BookForMe platform. 35
3.2 Login screen with role selection (Customer or Vendor). 37
3.3 Customer home: venue discovery by sport and area. 38
3.4 Booking flow: venue detail with date, court, and time slot selection (PKR pricing per slot). 38
3.5 In-app smart filter: AI chat parses Roman Urdu/English queries and returns filtered venues with available slots. 39
3.6 Community forum. 40
3.7 Direct messages. 40
3.8 Friends list. 40
3.9 Open matches: casual and ranked lobbies. 41
3.10 Ranked leaderboard with Elo ratings and tiers. 41
3.11 Vendor dashboard: daily stats, pending actions, slot operations, and upcoming bookings. 42
3.12 Live court grid calendar: per-court slot status with WhatsApp and walk-in booking sources. 42
3.13 WhatsApp AI receptionist: bilingual booking conversation returning available padel slots across multiple venues. 43
3.14 Admin control center: platform-wide stats, slot generation, and vendor moderation. 44
3.15 Admin vendor moderation: approve, suspend, or delete registered vendors. 44

4.1 High-Level System Architecture 46
4.2 Entity Relationship Diagram defining referential structures. 48

4.3 AI Booking & Payment Pipeline 50
4.4 Activity Diagram: Agent/WhatsApp Booking Flow 52
4.5 Activity Diagram: Mobile App Search & Book 53
4.6 Activity Diagram: Vendor Dashboard & Approvals 54

6.1 DistilBERT per-class classification report on the held-out test set . . 67
6.2 Confusion matrix for DistilBERT 68

List of Tables

1.1 BookForMe project timeline and deliverables. 16

2.1 Gap analysis: BookForMe versus existing dialogue and booking paradigms. 27

4.1 Domain Entity Descriptions 47

6.1 Intent distribution in the full BookForMe bilingual corpus (694 samples). 64
6.2 Hyperparameters used for all fine-tuning experiments. 65
6.3 Overall NLU model comparison (5-class intent classification). 66
6.4 AI Receptionist agent test case results. 69

1. Introduction

1.1 Problem Statement

Booking recreational and informal services in Pakistan is a fragmented and inefficient process. A customer who wants to reserve a futsal court for Friday evening has to find the vendor's WhatsApp number, ask about availability, wait for a manual reply that may take hours, negotiate a time and price, transfer an advance payment via JazzCash or bank transfer, and send a payment screenshot as confirmation. If the chosen slot was taken by another customer during the wait, the entire process restarts with a different vendor.

This friction is most visible in sports courts and similar slot-based venues in Karachi. No industry-wide booking infrastructure serves this segment. Existing digital solutions are narrow in scope, covering only sports apps or niche platforms, and they do not reflect the cross-category, bilingual booking behaviour of real users.

A user survey conducted by the team in Spring 2025 ($n > 50$) quantified this directly [3]:

- **72%** of respondents rely on WhatsApp or phone calls to make bookings.
- **81%** reported frustration with contacting multiple vendors to find an available slot.
- **68%** said they would switch to an app that could handle the conversation and booking on their behalf.

Research by MoldStud [22] corroborates that automated booking systems "increase efficiency, reduce human error, and improve customer satisfaction." Kumar et al. [15] further show that structured digital workflows reduce errors and missed reservations compared to manual methods. Chatterjee et al. [4] demonstrate through a Turf Booking System study that real-time slot availability and automated conflict resolution

improve both user experience and vendor management, though their solution remains domain-specific and does not extend to informal multi-category vendors or support autonomous agent-driven negotiation.

The vendor side mirrors this problem. Vendors manage incoming WhatsApp requests across multiple simultaneous conversations, maintain schedules in notebooks or digital calendars, and face double-bookings whenever they are offline or slow to reply. Small vendors lack the resources to build dedicated booking systems, causing missed revenue and a poor customer experience.

The core computer science challenge is: *how can an AI agent understand and autonomously execute bilingual natural-language booking requests in real time, while guaranteeing consistency in a distributed, multi-channel environment?* This breaks into four concrete sub-problems:

1. **Bilingual NLU:** Parse booking requests in both English and Roman Urdu, handling code-switching (e.g., "Mujhe Friday ko 7 baje ke baad paddle court chahiye under 6000 rupees, preferably in defence") and severe orthographic variation (*waqt*, *vakt*, *wkht*, and *tym* are all romanisations of the Urdu word for "time").

2. **Dialogue Management:** Maintain conversation state across asynchronous WhatsApp sessions, handle partial information, manage clarification loops, and route user intent to the correct backend action.

3. **Agentic Execution:** Autonomously check availability, hold slots atomically, verify payment proofs via OCR, and notify vendors without hallucinating actions or bypassing business rules.

4. **Concurrency Safety:** Prevent race conditions in a distributed NoSQL environment where multiple simultaneous requests may attempt to book the same slot.

1.2 Proposed Solution

BookForMe is a centralized, agentic AI-powered booking platform built to address all four sub-problems above. It provides two interaction modes for customers and a unified backend for vendors:

1. **React Native Mobile App:** A full-featured mobile application where customers browse vendors by category and location, view real-time availability, select and confirm slots, upload payment proof, and access a social layer including matchmaking, forums, and a leaderboard.

2. **WhatsApp AI Receptionist:** An autonomous agent running on the vendor's existing WhatsApp Business number. Customers message in plain English or Roman Urdu; the agent understands the request, checks availability in Firestore, holds the slot via Optimistic Concurrency Control, requests a payment screenshot, verifies the amount via OCR, and notifies the vendor for final approval. The vendor does not need to participate in the conversation at all.

Both clients share a single backend and Firestore database, meaning a slot booked via WhatsApp is immediately visible in the mobile app and on the vendor dashboard. Figure 1.1 gives a high-level overview of this architecture.

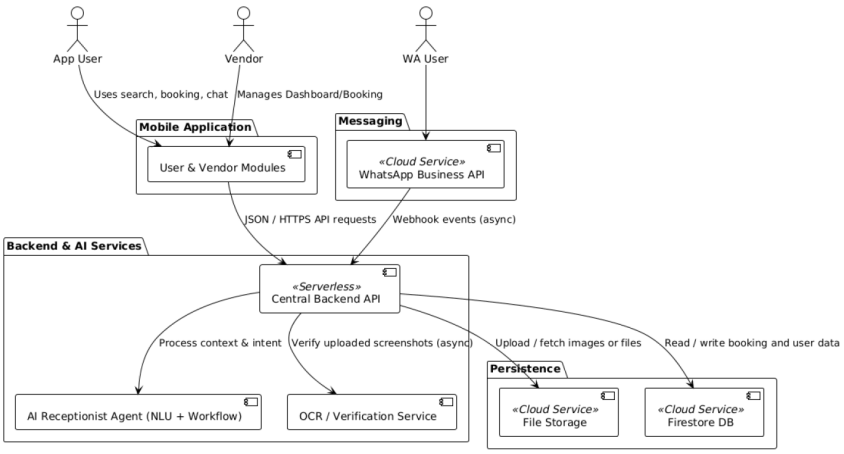


Figure 1.1: High-level overview of the BookForMe platform.

What sets BookForMe apart from existing tools is the combination of three capabilities in one platform: bilingual NLU tuned to the local mix of English and Roman Urdu; a fully autonomous agentic workflow over WhatsApp with transactional safety; and a social layer for player matchmaking and community across sports and other informal services.

1.3 Intended Users

BookForMe serves three distinct user groups:

App Users (Customers) Residents of Karachi looking to book informal services, predominantly sports-active young adults aged 18–35. They interact with the platform through the React Native mobile app, using either the AI chat filter for natural-language search or manual browsing with filters for price, location, and amenities. Social features such as player matching, leaderboards, and forums are central to this group.

WhatsApp Users (Casual Customers) Users who prefer not to install a separate app. They interact entirely through the vendor's existing WhatsApp Business number. The AI Receptionist handles their full booking journey from availability check to payment confirmation without requiring them to change how they communicate.

Vendors (Small Business Owners) Owners of futsal courts, padel courts, and similar informal services in Karachi. They interact with BookForMe through the Vendor Dashboard, which provides a consolidated booking calendar aggregating requests from both the app and WhatsApp, analytics, pending payment approvals, and WhatsApp Business account linking. Vendors need no technical expertise; onboarding requires only connecting their WhatsApp number.

1.4 Project Timeline and Deliverables

Table 1.1 shows the monthly project timeline across both Kaavish I (CS 491) and Kaavish II (CS 492) semesters.

Table 1.1: BookForMe project timeline and deliverables.

Month	Tasks	Deliverables
Sep–Oct 2025(Kaavish I)	Literature review; user survey; initial SRS; system architecture design; team role assignment; GitHub repository setup.	SRS v1; survey results; architecture diagram; project proposal.
November 2025	SRS v2 and SDS; initial bilingual dataset construction; low-fidelity UI wireframes; Firebase project and Firestore schema setup.	SRS v2; SDS; preliminary dataset (200 utterances); wireframes.
December 2025	Baseline NLU models (intent classification); Firestore CRUD APIs; React Native app skeleton (login, home, vendor listing).	Baseline NLU model; defined DB schema; minimal working booking system.
January 2026(Kaavish II)	Fine-tune NLU pipeline; LangGraph agent implementation; WhatsApp webhook integration; OCC slot locking; OCR payment verification.	Functional AI Receptionist; WhatsApp-to-Firestore data flow; enhanced user dashboard.
February 2026	Vendor dashboard (calendar, analytics, booking approval); social features (matchmaking, forum, leaderboard, chat); model retraining and error analysis.	Full vendor dashboard; active social features; optimised bilingual NLU model.
March 2026	End-to-end integration; unit and integration testing; performance benchmarking; UX testing with real users; mid-evaluation presentation.	Fully integrated platform; test reports; benchmark metrics; mid-eval presentation.
April 2026	Pilot testing with selected Karachi vendors; production deployment on Firebase and Render; final documentation and report.	Production-ready deployment; final report; presentation deck; submission package.

Kaavish I Deliverables:

- Project proposal (approved)

- SRS v1 and v2
- Software Design Specification (SDS)
- Literature review
- Preliminary bilingual dataset
- Low-fidelity wireframes

Kaavish II Deliverables:

- Functional AI Receptionist (WhatsApp agent, live demo)
- Full-stack React Native application (user app + vendor dashboard)
- Fine-tuned DistilBERT NLU model (90.5% accuracy)
- OCR payment verification module
- Social features (matchmaking, forum, leaderboard)
- End-to-end test suite and benchmark results
- Production deployment (https://jhat-to9p.onrender.com)
- This final report

1.5 Key Challenges

Roman Urdu Orthographic Variation Roman Urdu lacks a standardised orthography. A single word like *waqt* (time) may appear as *vakt*, *wkht*, *waqat*, or *tym* depending on the writer. Standard subword tokenisers (BPE, WordPiece) frequently fail to map these variants to a shared representation. We address this through targeted data augmentation and the use of multilingual models exposed to diverse romanisation patterns.

Low-Resource Bilingual Dataset No publicly available labelled dataset exists for English and Roman Urdu booking conversations. We constructed one from scratch using real vendor conversations, manual annotation, and LLM-driven synthetic generation. Starting from a small seed set introduces noise and class imbalance, requiring focal loss training and augmentation strategies.

Asynchronous Conversation State WhatsApp conversations are inherently asynchronous: users may reply hours later, change their mind mid-booking, or send the wrong payment screenshot. Unlike synchronous chatbot frameworks, our agent must persist full conversation state in Firestore and resume gracefully. LangGraph's cyclic state graph design addresses this, but requires careful state serialisation.

Concurrency and Double-Booking Multiple users may simultaneously request the same time slot. In a distributed NoSQL database like Firestore, unprotected writes allow race conditions. We implement Optimistic Concurrency Control (OCC) using Firestore atomic transactions with a `hold_expires_at` timestamp. Tuning the hold duration required careful calibration: too short causes false failures, too long unnecessarily ties up inventory.

LLM Latency Calls to the Groq API for natural language generation add latency per message. The total WhatsApp response time budget is 60 seconds (NFR-PERF). We minimise this by using the fine-tuned DistilBERT model for fast intent classification (47 ms on CPU) and invoking the LLM only for generation and edge cases, while caching frequent availability queries.

Prompt Injection Security Malicious users may attempt to trick the WhatsApp agent into bypassing payment requirements through crafted messages. The Neuro-Symbolic architecture prevents this: the LLM classifies intent, but all database mutations are executed by deterministic code nodes, so the LLM cannot directly issue database commands regardless of the input.

2. Literature Review

This chapter reviews the literature across five areas that underpin BookForMe: agentic AI systems; task-oriented dialogue and state tracking; bilingual and code-switched NLU; model selection and training for low-resource settings; and distributed booking systems with concurrency guarantees. Each section motivates the specific technical decisions made in the project and identifies the gaps in existing work that this project addresses.

2.1 From Chatbots to Autonomous Agents

2.1.1 Defining the Agentic Architecture

The term "agent" has been used broadly in software engineering and robotics, but its application to large language models (LLMs) represents a specific architectural shift. Wang et al. [34] propose a unified framework in which an LLM-based agent comprises four modules: *Profiling* (role and persona definition), *Memory* (short-term context and long-term persistent state), *Planning* (decomposing goals into subgoals and choosing actions), and *Action* (calling external tools and APIs to effect changes in the environment). The LLM functions as a coordinator that orchestrates a Perception $\rightarrow$ Reasoning $\rightarrow$ Action cycle, fundamentally distinguishing it from a passive text predictor.

Xi et al. [37] further distinguish agents from traditional conversational systems by their capacity to *initiate and manage workflows* rather than merely respond to prompts. A conventional chatbot can answer "Is the 5 PM slot available?"; our agent must answer "Book the 5 PM slot, lock it for ten minutes, verify the advance payment, and notify the vendor." The latter requires stateful read-write operations across multiple external systems, which is precisely the capability gap this work fills.

2.1.2 Cognitive Architectures: ReAct, PRACT, and Reflexion

Yao et al. [38] introduced the **ReAct** (Reasoning + Acting) paradigm, which interleaves chain-of-thought reasoning with external actions, enabling explicit planning traces such as:
Thought: User requests 5 PM padel. → *Action*: `get_availability("padel","2026-03-15","17:00")`. → *Observation*: Slot available. → *Action*: `create_booking(user, slot)`.

Liu et al. [19] extend this with the **PRACT** agent, adding a self-reflection step before executing irreversible actions. Before calling `create_booking()`, the agent verifies that the action is consistent with the user's latest stated intent, which is a critical safeguard where a confirmed booking constitutes a financial commitment.

Shinn et al. [32] demonstrate in **Reflexion** that verbal reinforcement and persistent state across iterations significantly improve agent reliability on multi-step tasks. The agent receives textual feedback from the environment (e.g., "payment amount incorrect") and updates its policy accordingly without gradient updates. This pattern directly informs our OCR rejection loop, where the agent re-prompts the user upon detecting an incorrect payment amount.

2.1.3 Orchestration Frameworks: LangChain and LangGraph

LangChain [20] provides a modular abstraction layer that decouples application logic from specific LLM providers, enabling composition of prompt templates, vector stores, and output parsers into unified pipelines. However, standard LangChain implementations rely on Directed Acyclic Graphs (DAGs), which cannot represent the iterative, self-correcting loops required for asynchronous WhatsApp conversations where users may reply late, change their mind, or submit incorrect payment proofs.

LangGraph [16] addresses this by replacing DAGs with *Cyclic State Graphs*: the agent's behaviour is modelled as a state machine where nodes represent actions and edges encode conditional logic. The framework provides loop control, state persistence

across invocations, and fault recovery, all of which are essential for BookForMe's multi-turn WhatsApp booking conversations. Shang et al. [30] confirm in AgentSquare that modular architectures separating Planning, Reasoning, and Memory are substantially more robust for complex tasks than monolithic prompt designs.

2.1.4 Multi-Agent Decomposition

Händler [10] proposes a multi-dimensional taxonomy of autonomous LLM-powered multi-agent architectures, showing that complex workflows benefit from assigning distinct roles to specialised sub-agents (intent parsing, verification, database updates, external communications) coordinated by a central orchestrator. Wu et al. [35] in AutoGen further argue that effective problem-solving requires cyclic loops: the ability to retry failed actions, request missing information, and self-correct rather than proceeding through purely linear pipelines. BookForMe adopts this multi-node, cyclic design in its LangGraph implementation.

2.2 Task-Oriented Dialogue and State Tracking

Task-Oriented Dialogue (TOD) systems have evolved from modular pipelines (NLU $\rightarrow$ Dialogue State Tracking $\rightarrow$ Policy $\rightarrow$ NLG) to end-to-end architectures. **SimpleTOD** [13] treats the entire dialogue as unified causal language modelling, jointly predicting belief states, system actions, and responses in a single autoregressive model. While effective on standard benchmarks (MultiWOZ), it requires large annotated corpora, which is a significant barrier for our domain where no pre-built dataset exists.

Shin et al. [31] address this data scarcity problem with **DS2** (Dialogue Summaries as Dialogue States), converting conversation history into templated natural-language summaries that map directly to structured slot representations. This summarisation-based approach is particularly useful for short, noisy WhatsApp messages where maintaining a full belief-state vector is impractical.

2.2.1 Why Separate NLU and NLG

End-to-end TOD systems that jointly handle understanding and generation perform well when large in-domain corpora exist. Peng et al. [25] in **Soloist** demonstrate that a unified generative model can handle full dialogue turns at scale through transfer learning and machine teaching, but this approach still requires thousands of task-specific annotated dialogues and introduces non-determinism in every response, including slot fills and booking decisions. For a production booking system where incorrect slot data leads to real financial transactions, non-determinism in the understanding stage is unacceptable.

This motivates BookForMe's two-model split: a deterministic, fine-tuned classifier (DistilBERT) for intent routing, and a large generative model (Qwen 3 32B via Groq) only for surface-level natural language generation of responses. The classifier's output is typed and bounded; the generator never touches database state directly. This is consistent with the Neuro-Symbolic architecture recommended by Händler [10] and Wu et al. [35].

2.3 Bilingual and Code-Switched NLU

2.3.1 Roman Urdu Challenges

Processing Roman Urdu introduces challenges largely absent from standard multilingual NLP. Bhat et al. [2] classify Roman Urdu as exhibiting severe *orthographic variation*: a single word can be rendered in multiple phonetically plausible romanisations by different writers (e.g., *waqt*, *vakt*, *wkht*, *waqat*, and *tym* all mean "time"). Standard subword tokenisers (BPE, WordPiece) trained on formal corpora frequently fail to map these variants to a shared semantic representation. A normalisation step or a model pre-exposed to informal text is therefore necessary before intent classification.

2.3.2 Cross-Lingual Transfer and Adapters

Yu et al. [39] address cross-lingual dialogue state tracking via contrastive learning, aligning slot embeddings across languages so that semantically equivalent slots in different languages map to nearby vector representations. Wu et al. [36] show that parameter-efficient adapter methods can transfer pretrained models to code-switched contexts without full retraining, significantly reducing data and compute requirements.

The Qwen2.5 technical report [27] demonstrates strong multilingual instruction-following capability, validating its use for generating bilingual booking responses in English and Roman Urdu where tone, code-switching, and natural phrasing are all important.

2.3.3 Pretrained Models for Classification

Devlin et al. [7] introduced **BERT**, establishing bidirectional masked language modelling as the dominant pretraining paradigm. The multilingual variant (mBERT) was pretrained on 104 languages including Urdu, providing limited but useful cross-lingual transfer. Conneau et al. [6] demonstrate that **XLM-RoBERTa**, trained on a substantially larger multilingual corpus with improved data filtering, outperforms mBERT on cross-lingual classification tasks.

Sanh et al. [28] show that **DistilBERT**, a distilled 66M-parameter version of BERT, retains 97% of BERT's NLP benchmark performance while being 40% smaller and 60% faster at inference. Knowledge distillation [12] achieves this by training a compact student model to replicate the soft output distribution of a larger teacher, preserving most of the representational quality at a fraction of the compute cost. For a latency-sensitive production system like BookForMe, where intent classification must complete in under 50 ms per message, this speed-accuracy tradeoff is the primary motivation for choosing DistilBERT over larger alternatives.

2.3.4 Data Augmentation for Low-Resource Settings

Given the absence of any pre-built bilingual booking dataset, data augmentation is essential. Hou et al. [14] show that back-translation and contextual augmentation improve slot-filling F1 by 6–17% and intent accuracy by up to 12% in booking domains with fewer than 500 annotated utterances. An et al. [1] demonstrate that controllable T5-based generation yields 8–18% absolute gains in intent classification under 10-shot conditions for restaurant and hotel reservation tasks. He et al. [11] in **SynthDST** show that fully synthetic LLM-generated dialogues can surpass real-data baselines, achieving 4–12% higher Joint Goal Accuracy on MultiWOZ when human annotations are extremely limited. BookForMe's augmentation strategy follows these findings directly.

Focal loss [18], originally introduced for object detection, is effective for training on class-imbalanced datasets: it down-weights easy, well-classified examples and focuses training signal on the harder class boundaries. Given the imbalance in our 5-class corpus (39.9% inquiry vs. 5.3% greeting), focal loss is more appropriate than standard cross-entropy.

2.4 Retrieval, Planning, and Tool Use

Schick et al. [29] demonstrated in **Toolformer** that LLMs can learn to invoke external APIs through self-supervised training, deciding autonomously when and how to call tools. Patil et al. [24] (**Gorilla**) and Qin et al. [26] (**ToolLLM**) extend this to controlled invocation of structured APIs, providing the theoretical basis for BookForMe's `get_availability()`, `create_booking()`, and `get_services()` tool set.

Lewis et al. [17] introduced **Retrieval-Augmented Generation (RAG)**, conditioning generation on dynamically retrieved documents. In BookForMe, the agent retrieves up-to-date vendor availability and pricing from Firestore rather than relying on memorised parametric knowledge, which is critical for a real-time booking system.

Chen et al. [5] present adaptive retrieval strategies where the agent decides when retrieval is necessary, minimising unnecessary database queries and reducing latency.

2.5 Payment Verification via Computer Vision

A distinguishing feature of BookForMe is its automated payment screenshot verification via OCR, eliminating the need for manual receipt checking by vendors. Memon et al. [21] provide a comprehensive review of OCR systems, noting that modern deep learning-based OCR achieves high accuracy on printed and near-printed text such as bank transfer screenshots, where fonts are consistent and the background is predictable. For edge cases such as blurry images or unusual receipt formats, their analysis recommends requesting a cleaner image rather than forcing a low-confidence extraction, which aligns exactly with our fallback strategy.

The Gemini multimodal model [9] extends this capability to structured prompting: given a screenshot and a question ("what amount was transferred?"), the model returns a specific extracted value rather than a general description. This approach is more reliable for our use case than training a dedicated OCR model, as it leverages a large pretrained vision backbone without requiring any labelled payment screenshot data.

2.6 Distributed Booking and Concurrency Safety

Mong'are [23] provides a thorough analysis of idempotency in APIs and distributed systems, specifically addressing the double-booking problem that arises when concurrent requests modify shared mutable state. The recommended approach is idempotent APIs combined with Optimistic Concurrency Control (OCC) or distributed locking. BookForMe's implementation directly follows this recommendation, using Firestore atomic transactions with a `hold_expires_at` timestamp to implement OCC.

Singh et al. [33] demonstrate the effectiveness of structured state management for dialogue policy in the NJFun system, validating the importance of separating

dialogue state from the booking transaction state, which is exactly the separation BookForMe maintains between the LangGraph `AgentState` and the Firestore slot document.

2.7 Social Ranking: Elo Rating Systems

The BookForMe social layer includes a competitive matchmaking system using Elo ratings. The Elo system [8], originally developed for chess, assigns each player a numerical rating updated after each match based on the expected versus actual outcome, with the magnitude of the update weighted by the opponent's relative strength. This produces a self-calibrating ranking that accounts for the quality of opponents rather than simply counting wins. It is widely adopted in competitive online games (e.g., League of Legends, Chess.com) and is well understood by sports players, making it an appropriate choice for the casual and competitive match lobbies in BookForMe's social hub.

2.8 Novelty and Gap Analysis

Table 2.1 summarises the specific gaps that BookForMe addresses relative to the existing literature and systems.

Table 2.1: Gap analysis: BookForMe versus existing dialogue and booking paradigms.

System	Focus	Tool Use	Roman Urdu	Async Mem.	Txn. Safety
GPT / General LLM	General chat	✓	✓(partial)	✗	✗
SimpleTOD	Research datasets	✗	✗	✗	✗
ReAct Agents	Web automation	✓	✗	✓(limited)	✗
Domain-specific apps	Single category	✗	✗	✗	✓(partial)
BookForMe	**WA Booking**	✓	✓	✓	✓

The primary research gaps addressed are:

- **Roman Urdu code-switching:** Existing DST and NLU research does not target informal WhatsApp-style Roman Urdu, presenting gaps in language modelling and orthographic normalisation for this dialect.
- **Transactional booking guarantees:** While agent tool-use is well studied, very few works address concurrency, idempotency, and transactional safety in real-world booking systems.
- **Asynchronous conversation flow:** Most TOD frameworks assume synchronous interaction. WhatsApp requires persistent memory and session resumption across arbitrary time delays.
- **Evaluation for informal-economy SMEs:** Agent evaluation literature does not address booking systems in informal economies where reliability, low latency, and no-infrastructure vendor onboarding are essential constraints.

BookForMe synthesises the reviewed literature into a practical LLM-driven booking agent that is novel precisely because it operates at the intersection of all four gaps listed above.

3. Software Requirement Specification (SRS)

3.1 Introduction

3.1.1 Purpose

This document defines the requirements for the **BookForMe** platform. The purpose of BookForMe is to address inefficiencies and fragmentation in booking sports courts and similar slot-based venues within Karachi. The system replaces unstructured communication and manual record-keeping with a centralized, intelligent, and reliable platform.

It provides a streamlined booking interface for end-users alongside a social community layer for players. Vendors receive tools to manage bookings and availability across all channels, including an autonomous AI assistant on WhatsApp, reducing operational friction and revenue loss from missed or mishandled requests.

3.1.2 Scope

The BookForMe system comprises four primary components operating together:

1. **User-facing mobile application:** A React Native (Expo) app through which customers discover venues, view real-time slot availability, place bookings, and participate in social features including friends, direct messaging, forums, matchmaking, and a ranked leaderboard.

2. **Vendor dashboard:** A vendor-facing React Native interface to register, manage a business profile (services, pricing, operating hours), view a consolidated booking calendar with live slot status, approve or reject pending payments, and connect a WhatsApp Business account.

3. **AI agent backend:** The intelligent engine built in Python (FastAPI and LangGraph). It receives WhatsApp traffic via webhooks, performs bilingual (Roman Urdu/English) natural language understanding using Groq (Qwen 3 32B), manages stateful conversational booking workflows, enforces concurrency control via Firestore transactions, and coordinates payment proof and vendor approval flows.

4. **Admin control panel:** A platform-level administrative interface for platform governance: vendor registration approval and moderation, slot generation, stale lock release, and oversight of pending payments across all vendors.

3.2 Functional Requirements

3.2.1 Module: User (Booking & Discovery)

- `FR-CUST-01`: The system shall allow users to browse and search registered vendors by service category (e.g., Padel, Futsal, Cricket) and location area.
- `FR-CUST-02`: The system shall provide filtering options (e.g., price, area, sport type).
- `FR-CUST-03`: The system shall display vendor profiles including name, location, services offered, and pricing.
- `FR-CUST-04`: The system shall present a real-time live court grid showing available, held, and booked slots per court per time block.
- `FR-CUST-05`: The system shall allow users to select an available time slot and confirm a booking.
- `FR-CUST-06`: The system shall confirm successful bookings to the user via the app interface.

3.2.2 Module: User (AI Chat & Discovery)

- **Smart filter** (FR-CUST-07): The system shall provide an in-app AI chat interface that parses natural-language queries in English or Roman Urdu (e.g., "Aaj Raat DHA mei konsa padel available hei under 2000") and returns filtered venue results with available slots.
- `FR-CUST-08`: The in-app AI assistant shall return matching venues, available slot times, and pricing in a single response.

3.2.3 Module: User (Social & Community)

- `FR-SOC-01`: The system shall provide user registration and login with role selection (Customer or Vendor).
- `FR-SOC-02`: The system shall allow users to create and manage a public profile.
- `FR-SOC-03`: The system shall provide a friend system: send, accept, and deny friend requests.
- `FR-SOC-04`: The system shall allow users to send and receive private direct messages with friends.
- `FR-SOC-05`: The system shall provide a community forum where users can create, view, like, and comment on public posts.

3.2.4 Module: User (Matchmaking & Ranking)

- `FR-MATCH-01`: The system shall allow users to create and join open matches (Casual or Ranked) for sports such as Padel, Futsal, and Cricket.
- `FR-MATCH-02`: The system shall calculate and update user Elo ratings based on ranked match outcomes.
- `FR-MATCH-03`: The system shall display a public leaderboard of top-ranked players, filterable by sport, showing rank, points, and tier (e.g., Silver).

3.2.5 Module: Vendor

- `FR-VEND-01`: The system shall provide secure registration and login for vendors, subject to admin approval before activation.
- `FR-VEND-02`: Vendors shall be able to create and manage their business profile (services, pricing, operating hours).
- `FR-VEND-03`: Vendors shall be able to view a live court grid calendar showing all bookings by court, time, and booking source (WhatsApp or walk-in).
- `FR-VEND-04`: The vendor dashboard shall display daily revenue, bookings count, pending action items, and upcoming unbooked slots.
- `FR-VEND-05`: The system shall provide an interface for vendors to generate availability slots and manage bookings.
- `FR-VEND-06`: The vendor dashboard shall provide an interface to review and approve or reject pending payments.
- `FR-VEND-07`: The system shall allow vendors to connect their WhatsApp Business account to activate the AI receptionist.

3.2.6 Module: AI Agent (WhatsApp Receptionist)

- `FR-AGENT-01`: The system shall receive inbound WhatsApp messages via a configured Meta webhook.
- `FR-AGENT-02`: The system shall send outbound WhatsApp replies (slot listings, confirmations, rejections) via the WhatsApp Business API.
- `FR-AGENT-03`: The system shall process inbound messages in English and Roman Urdu to extract booking intent and entities (service, date, time, area).
- `FR-AGENT-04`: The system shall maintain full conversational context (dialogue state) for ongoing WhatsApp interactions, persisted in Firestore across delays.

- `FR-AGENT-05`: Based on NLU output and real-time availability, the system shall present matching slots to the user and accept confirmation.
- `FR-AGENT-06`: The system shall implement Optimistic Concurrency Control via Firestore transactions to prevent double-booking across all channels simultaneously.
- `FR-AGENT-07`: After a user confirms a booking, the system shall request payment proof (screenshot) and set the booking to a hold state with an expiry window.
- `FR-AGENT-08`: Upon receiving the screenshot, the system shall update the booking to pending approval, attach the image, and notify the vendor dashboard.
- `FR-AGENT-09`: Upon vendor approval, the system shall update the booking status to confirmed and send a confirmation message with a booking reference to the user on WhatsApp.

3.2.7 Module: Admin

- `FR-ADMIN-01`: The system shall provide a platform administrator interface displaying pending vendors, pending payments, locked slots, and total active vendors.
- `FR-ADMIN-02`: The system shall allow admins to approve, suspend, or delete vendor accounts.
- `FR-ADMIN-03`: The system shall allow admins to generate availability slots across all vendors for a configurable window (e.g., next 14 days).
- `FR-ADMIN-04`: The system shall allow admins to release stale slot locks caused by abandoned booking sessions.

3.2.8 Use Case Diagrams

Figure 3.1 shows the high-level system overview. The use case interactions are summarized below:

- **Customer** uses the mobile app to log in, discover venues, book slots, use the AI chat filter, and access social features (forum, friends, matches, leaderboard).
- **WhatsApp user** interacts with the AI receptionist: sends booking requests in English/Roman Urdu, confirms a slot, submits payment proof, and receives a confirmation.
- **Vendor** uses the vendor dashboard to manage their profile, view the live court calendar, handle payment approvals, and link their WhatsApp account.
- **Admin** uses the admin panel to approve vendors, manage slots, and oversee payments.

3.3 Non-Functional Requirements

3.3.1 Performance

- **Performance** (NFR-PERF): Key app screens (vendor search, live court grid) shall load within 30 seconds on a stable mobile connection.
- **NLU processing** (NFR-PERF): The AI agent NLU processing for a WhatsApp message shall complete within 60 seconds.
- **Response time** (NFR-PERF): Total time from receiving a WhatsApp availability query to sending the initial response shall not exceed 60 seconds.
- **Throughput** (NFR-PERF): The WhatsApp webhook shall handle at least four simultaneous incoming requests without failure.

3.3.2 Security

- **Authentication** (NFR-SEC): All user and vendor access shall be protected via Firebase Authentication.

- **API key security** (NFR-SEC): All external API keys (WhatsApp, cloud AI providers) shall be stored as environment variables, not hardcoded.
- **Webhook verification** (NFR-SEC): The WhatsApp webhook shall verify all incoming requests using the Meta verify token.
- **Encryption** (NFR-SEC): All data transmission between clients, the backend, and external APIs shall use HTTPS/TLS.

3.3.3 Reliability

- **Uptime** (NFR-REL): The system shall target 99.0% uptime during peak operating hours.
- **Fault tolerance** (NFR-REL): The AI agent shall handle external API errors gracefully, logging them without crashing the main service.

3.3.4 Scalability

- **Cloud scaling** (NFR-SCAL): The backend shall support scaling via cloud deployment to handle increased load.
- **Concurrent users** (NFR-SCAL): The Firestore schema shall support at least 100 concurrent users without significant degradation.
- **Vendor capacity** (NFR-SCAL): The system shall support at least 10 active vendors without degradation.

3.3.5 Usability

- **Booking efficiency** (NFR-USAB): A new user shall be able to complete a booking in fewer than eight primary steps from the home screen.

- **Dashboard efficiency** (NFR-USAB): Key vendor dashboard functions (calendar, pending approvals) shall be accessible within six taps from the main dashboard.
- **Response clarity** (NFR-USAB): WhatsApp agent responses shall be clear and appropriate in both English and Roman Urdu.

3.4 System Block Diagram

Figure 3.1 presents the high-level system block diagram for the BookForMe platform, showing the frontend layer (customer app, vendor dashboard, admin panel), the backend API and AI agent layer, and external services (WhatsApp Business API, Firebase, Firestore).

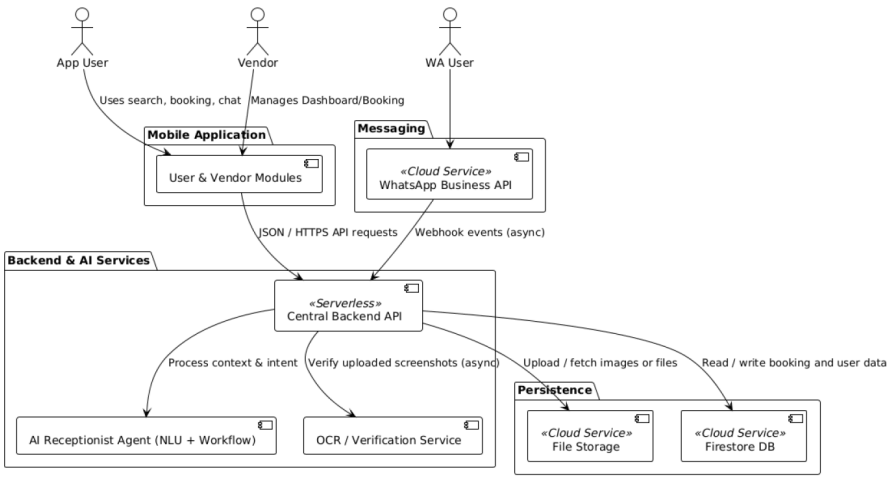


Figure 3.1: System block diagram for the BookForMe platform.

3.4.1 Component descriptions

- **Customer mobile app** — React Native (Expo). Handles discovery, booking, AI chat filter, and all social features. Communicates with the backend via HTTPS APIs.

- **Vendor dashboard** — React Native. Profile management, live court calendar, pending payment review, and WhatsApp account linking.
- **Admin panel** — React Native. Platform governance: vendor approval/moderation, slot generation, stale lock release, payment oversight.
- **AI agent backend** — Python/FastAPI with LangGraph. Manages the WhatsApp webhook conversation loop, bilingual NLU via Groq, stateful booking workflow, and Firestore-backed concurrency control (OCC).
- **Cloud Firestore** — Central NoSQL store for profiles, slots, bookings, conversation state, social data, and payment records.
- **WhatsApp Business API** — Inbound webhooks and outbound messaging channel for the AI receptionist.
- **Firebase Authentication** — Identity management for customers, vendors, and admins.

3.5 System Screenshots

The following screenshots from the deployed BookForMe application illustrate the key user-facing and vendor-facing interfaces described in the functional requirements above.

3.5.1 Authentication

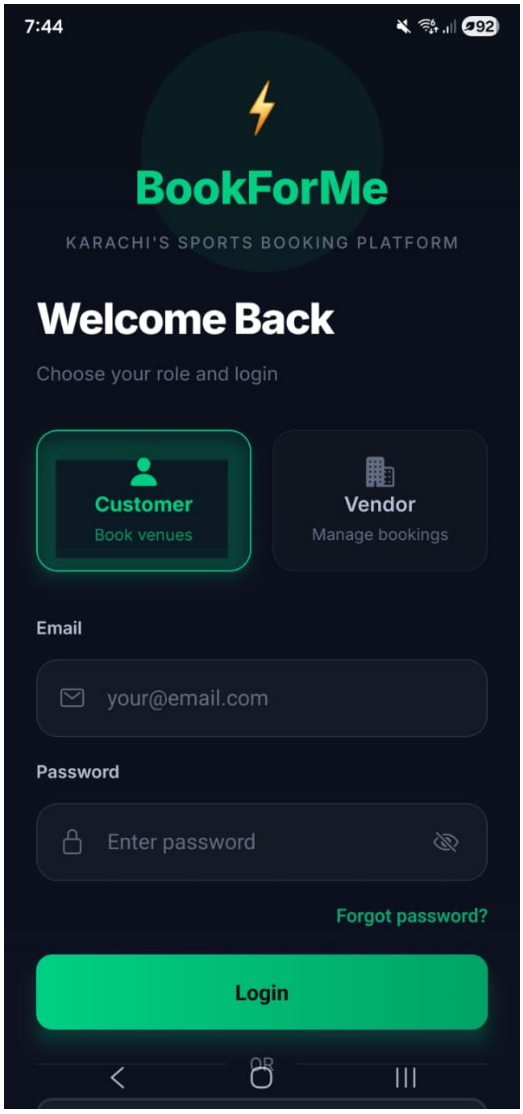


Figure 3.2: Login screen with role selection (Customer or Vendor).

3.5.2 Customer application

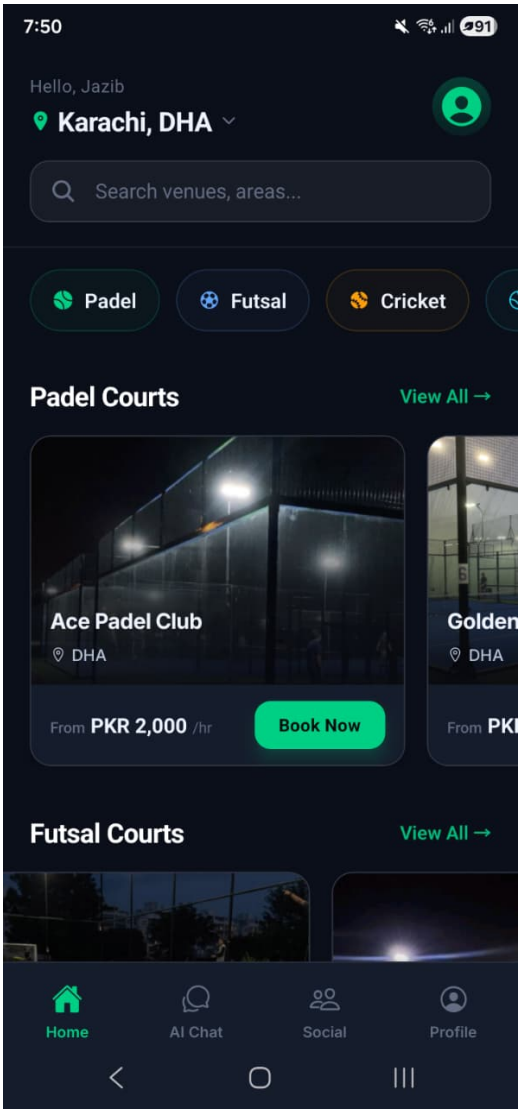


Figure 3.3: Customer home: venue discovery by sport and area.

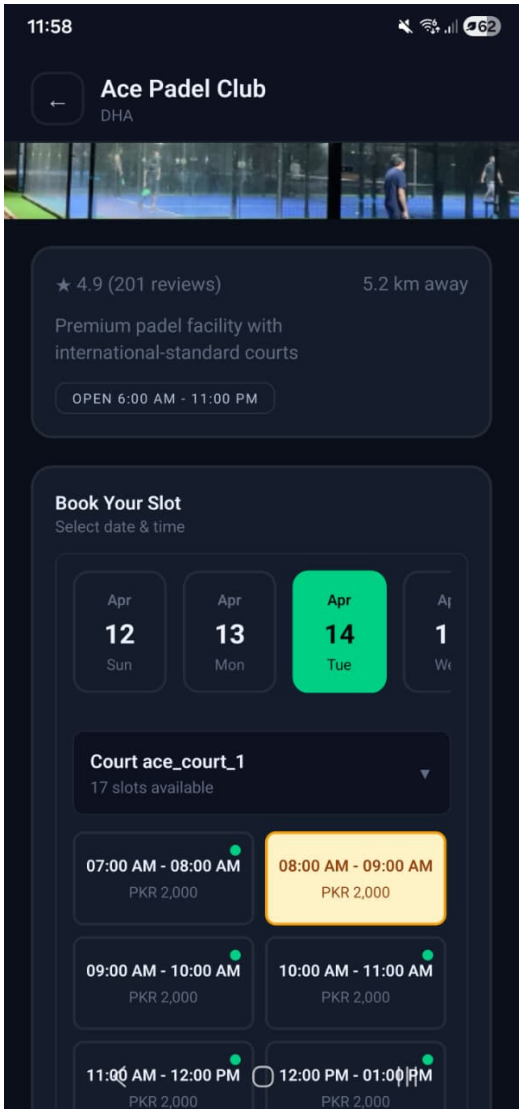


Figure 3.4: Booking flow: venue detail with date, court, and time slot selection (PKR pricing per slot).

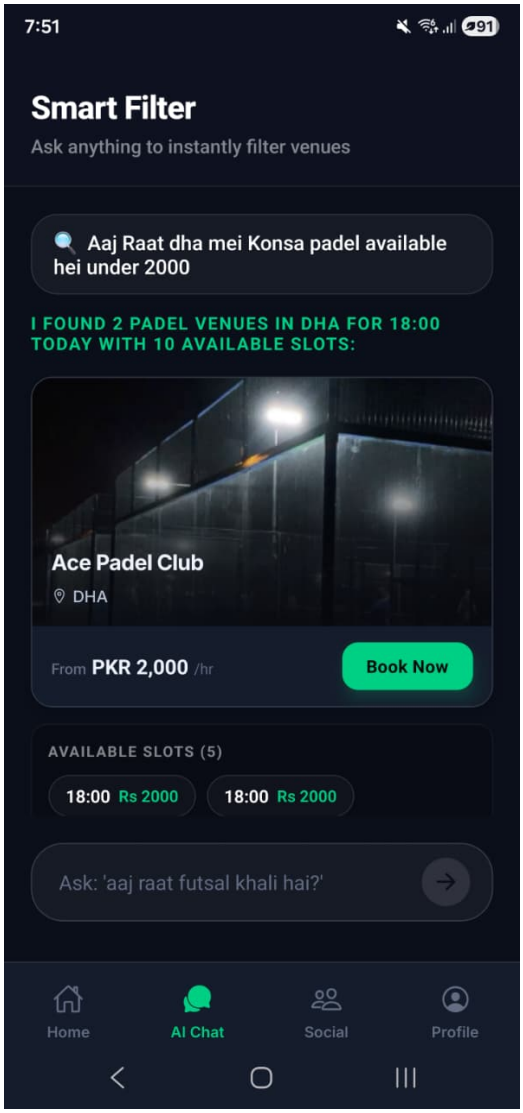


Figure 3.5: In-app smart filter: AI chat parses Roman Urdu/English queries and returns filtered venues with available slots.

3.5.3 Social hub

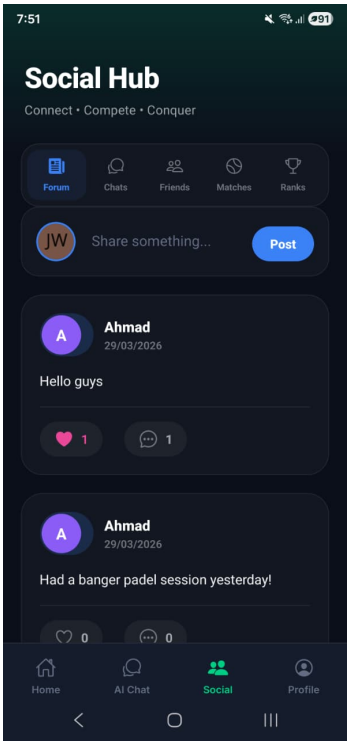


Figure 3.6: Community forum.

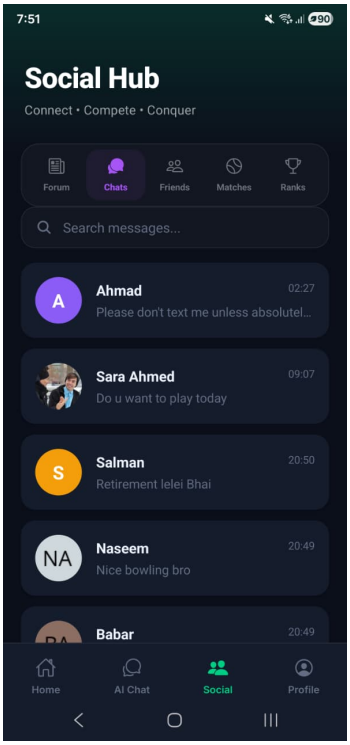


Figure 3.7: Direct messages.

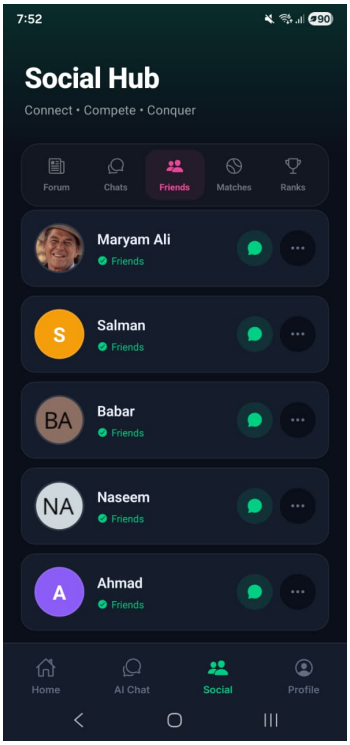


Figure 3.8: Friends list.

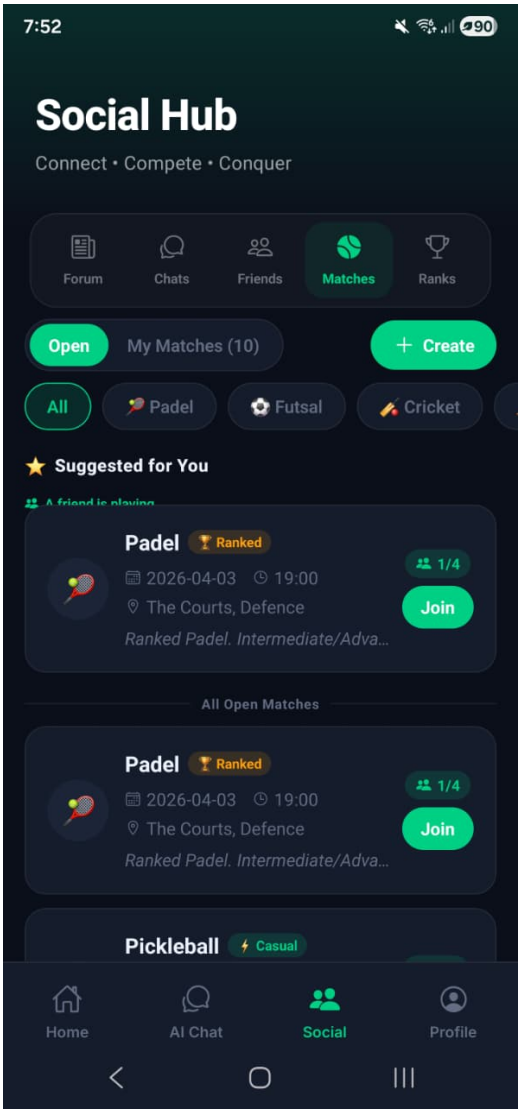


Figure 3.9: Open matches: casual and ranked lobbies.

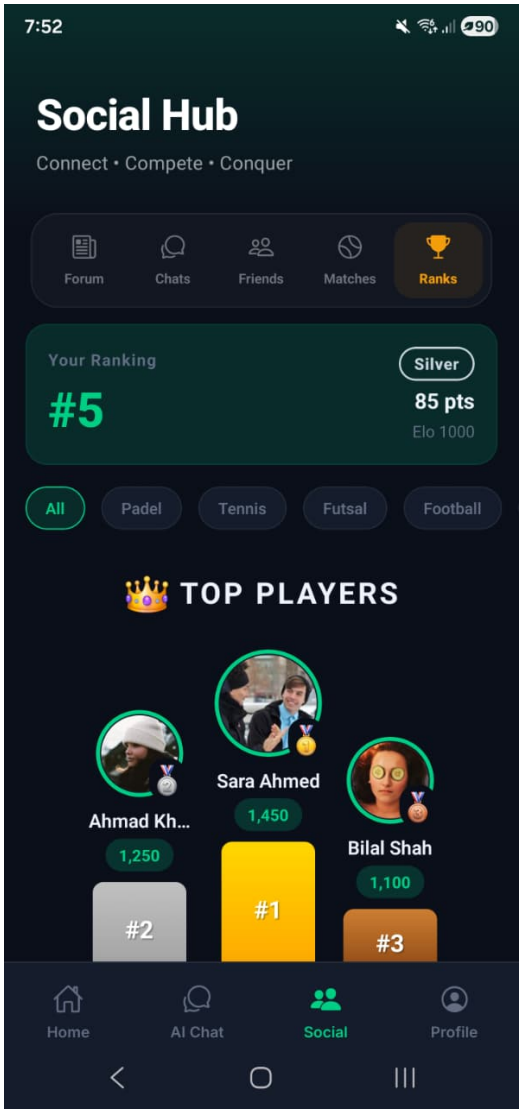


Figure 3.10: Ranked leaderboard with Elo ratings and tiers.

3.5.4 Vendor dashboard

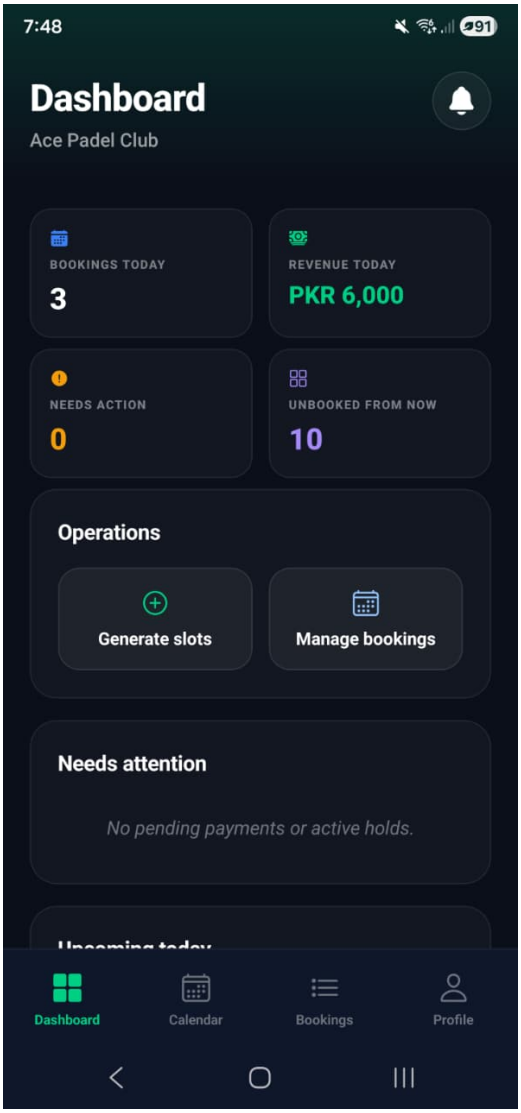


Figure 3.11: Vendor dashboard: daily stats, pending actions, slot operations, and upcoming bookings.

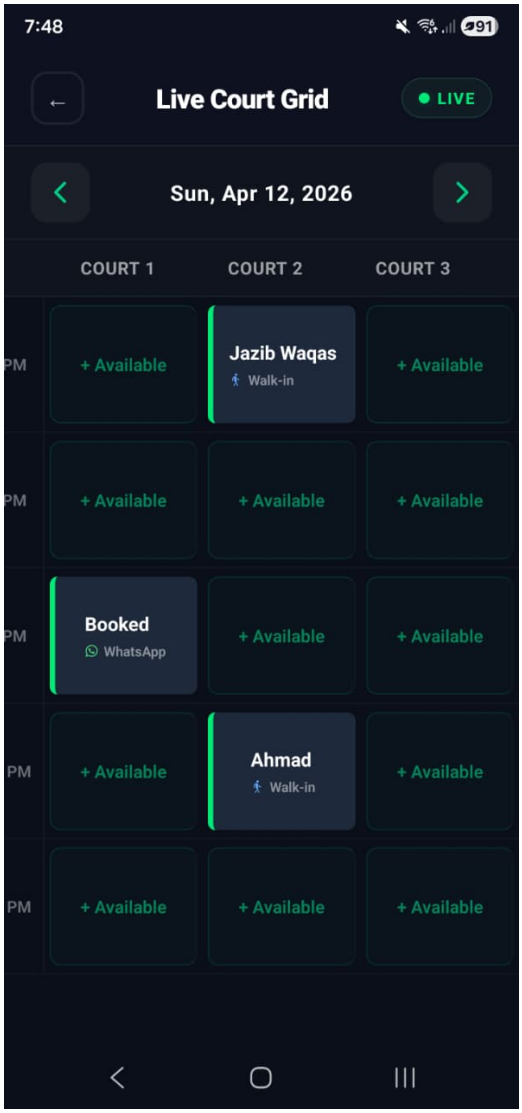


Figure 3.12: Live court grid calendar: per-court slot status with WhatsApp and walk-in booking sources.

3.5.5 AI receptionist (WhatsApp)

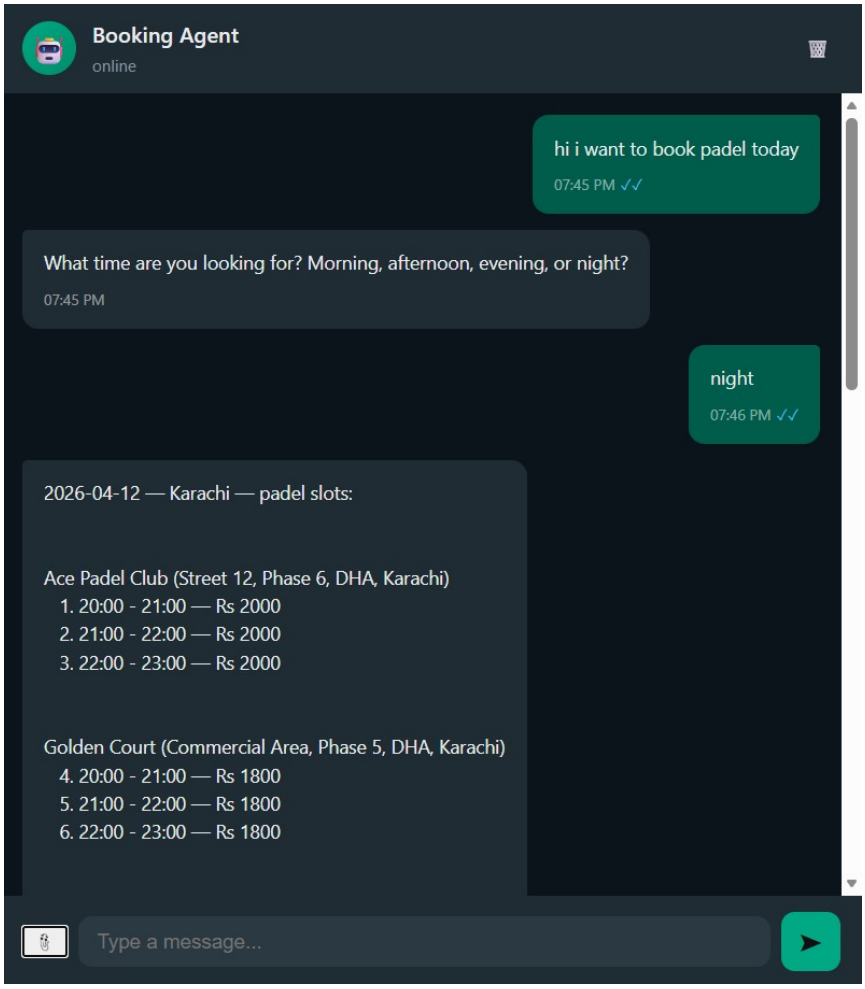


Figure 3.13: WhatsApp AI receptionist: bilingual booking conversation returning available padel slots across multiple venues.

3.5.6 Admin control panel

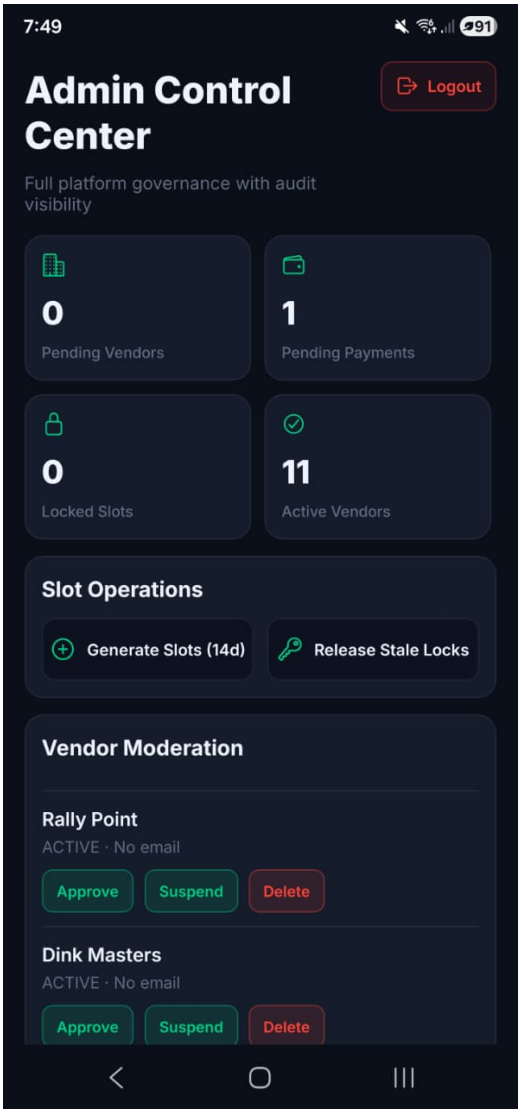


Figure 3.14: Admin control center: platform-wide stats, slot generation, and vendor moderation.

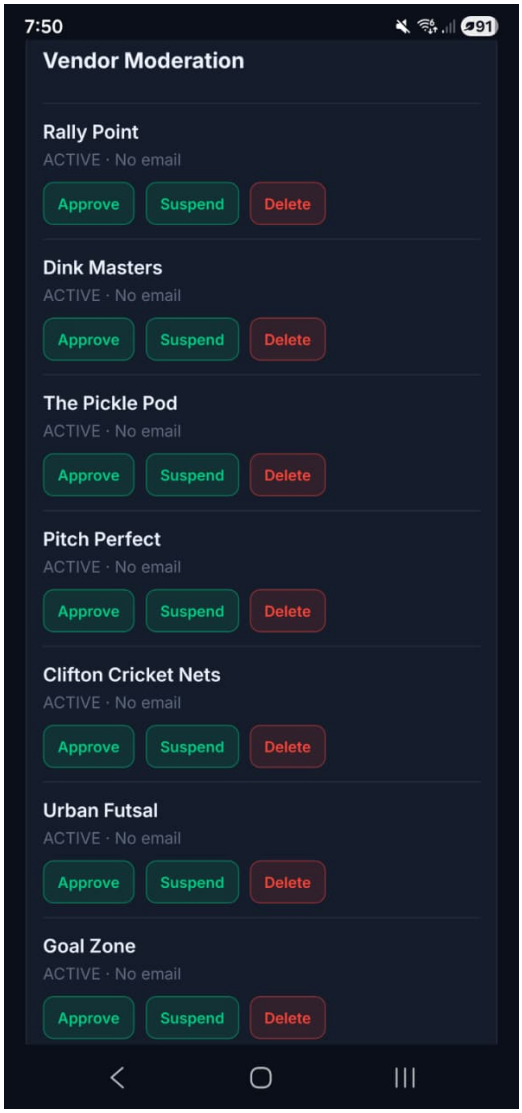


Figure 3.15: Admin vendor moderation: approve, suspend, or delete registered vendors.

4. Software Design Specification (SDS)

This chapter details the BookForMe system's architectural patterns, data structures, and the workflow logic required to build a robust service booking application. It serves as the technical blueprint for the implementation described in the following chapters.

4.1 System Architecture

4.1.1 Architectural Design

The system follows a **Service-Oriented Architecture (SOA)** with a clear separation between the Presentation, Business Logic, and Persistence layers. Both the Rich Client and the Conversational Client converge on a single backend and Google Cloud Firestore database, ensuring a "Single Source of Truth." Dependencies are strictly unidirectional: Clients depend on the Backend, and the Backend depends on the Database.

4.1.2 System Architecture Diagram

The diagram below illustrates the high-level data flow across the system's three tiers.

- **Synchronous API Flow:** The Mobile Application and Vendor Dashboard communicate directly with the Central Backend API via JSON/HTTPS requests.
- **Asynchronous Event Flow:** The WhatsApp API operates via webhooks, triggering an asynchronous event routed to the LangGraph AI Receptionist.

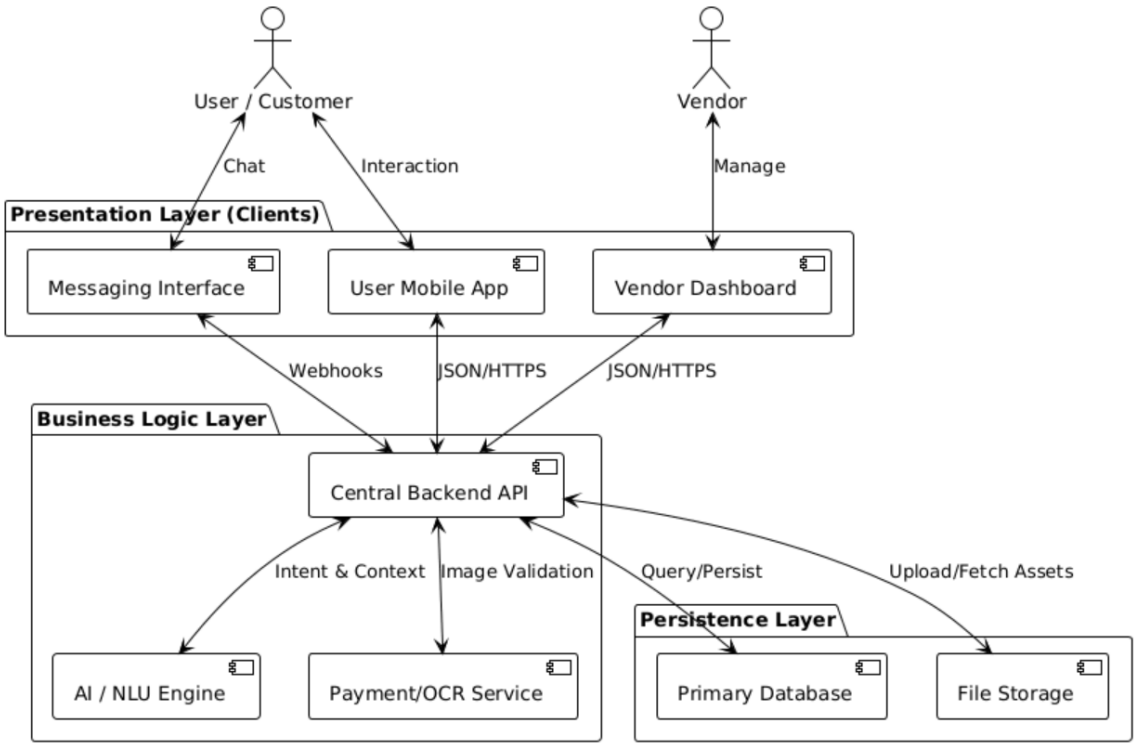


Figure 4.1: High-Level System Architecture

4.2 Data Model (Data Design)

4.2.1 Database Strategy

Data persistence is handled by **Google Cloud Firestore**. We employ a "Reference-Based" schema strategy to balance the flexibility of NoSQL with the need for structured relationships. The design strictly separates the transactional domain (vendors, slots, payments) from social or profile elements to ensure data integrity.

4.2.2 Entity Descriptions

Table 4.1 describes the primary data collections deployed in Firestore.

Table 4.1: Domain Entity Descriptions

Collection	Key Attributes	Description & Purpose
users	phone, role, stats	Maintains core identity, authentication references, and gamification profiles.
vendors	operating_hours, revenue	Represents the business entity and operational constraints.
services & resources	pricing, capacity	Sellable units (e.g., Padel) mapped to specific real-world physical assets (e.g., Court 1).
slots	status, hold_expires_at	The core inventory ledger representing blockable time intervals.
payments	screenshot_url, ocr_verified_amount	Audit trail of transactions, separated to ensure OCR verification operates safely before bookings are confirmed.

4.2.3 NoSQL Schema Diagram

The Entity Relationship Diagram delineates references between entities, mapping out the foreign keys (e.g., `vendor_id`) that correlate disparate collections.

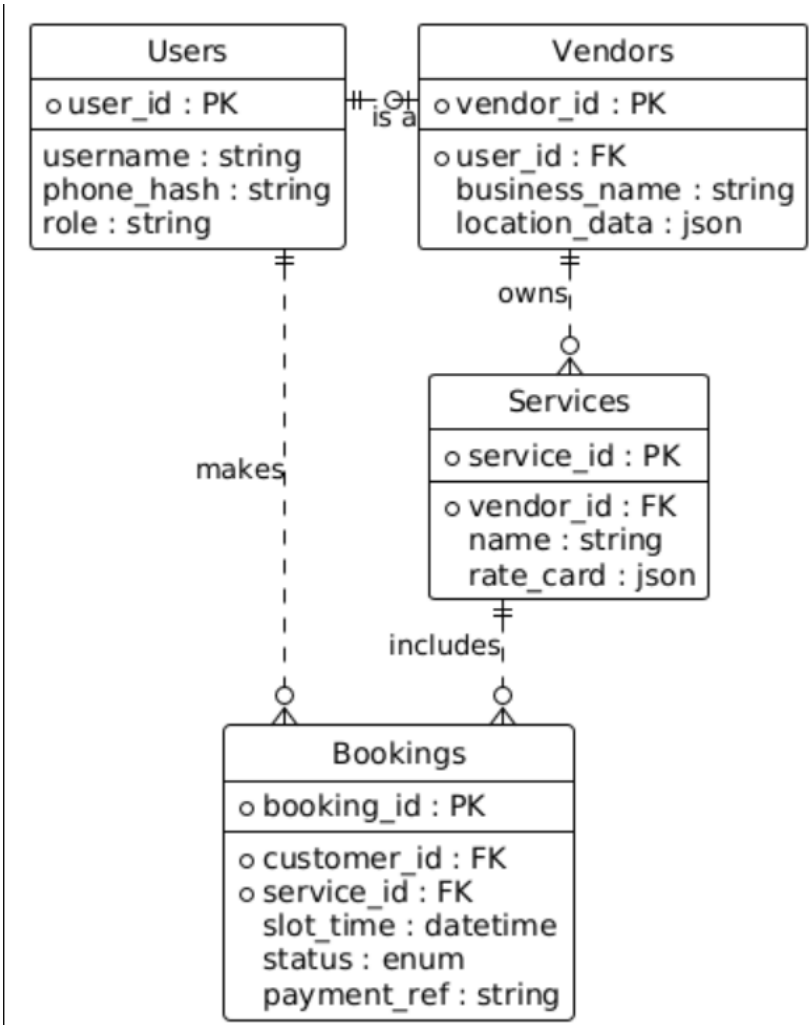


Figure 4.2: Entity Relationship Diagram defining referential structures.

4.3 Agentic Booking Process Pipeline

A core functionality is the AI-driven booking process managed by a LangGraph temporal state machine. It manages stateful conversations, enforcing a strict validation pipeline.

4.3.1 Pipeline Logic

1. **Ingest & Negotiation:** The Agent loops with the user to clarify intents (e.g., `GREETING`, `TRANSACTION`) and specific entities (date, time, service).

2. **Transaction (Locking):** Using OCC, the system creates a `HOLD` booking state with a 10-minute expiry window.

3. **Intelligent Gatekeeper:** Gemini Vision OCR extracts the payment screenshot amount. If invalid, the Agent auto-rejects quietly, asking for a re-upload.

4. **Human-in-the-Loop:** Valid receipts reach the Vendor Dashboard for a final localized authorization.

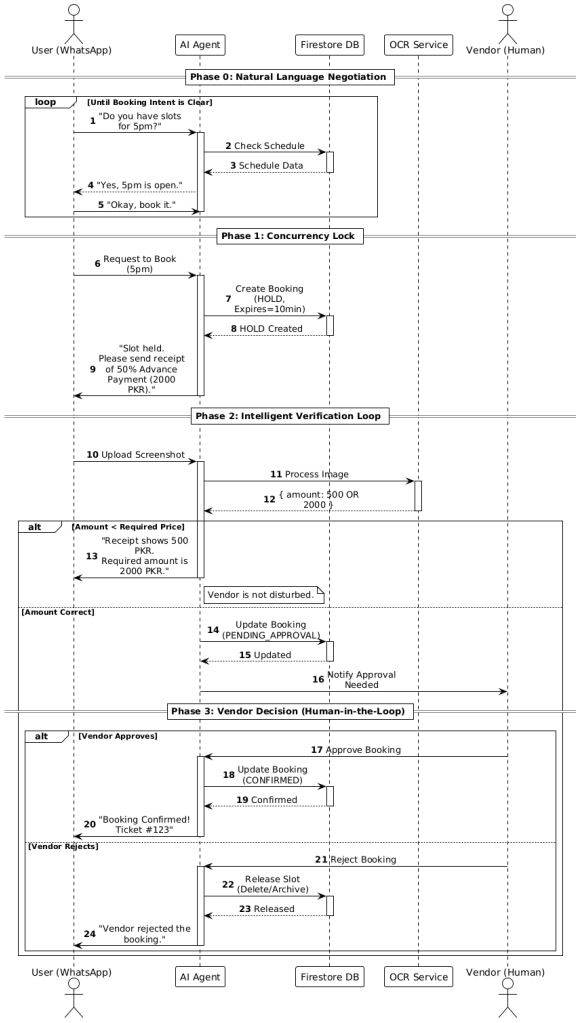


Figure 4.3: AI Booking & Payment Pipeline

4.4 Technical Details & Workflows

4.4.1 Key Algorithms

Algorithm 1: Intent Parsing & Action Routing

Purpose: To translate unstructured natural language into deterministic database actions without hallucination.

- **Input:** Raw WhatsApp Message string + User Session State.
- **Logic:** Qwen 3 32B classifies intent. The state machine "routes" this to a deterministic Python node ensuring the LLM does not generate database queries directly.
- **Output:** Validated JSON Payload or Clarification Prompt.

Algorithm 2: Optimistic Concurrency Control (OCC)

Purpose: To prevent race conditions (double bookings) in a distributed environment.

- **Input:** `service_id`, `time_slot`, `user_id`.
- **Logic:**
  1. **Read:** Fetch the current slot status.
  2. **Verify:** Ensure status is `AVAILABLE`.
  3. **Write:** Atomically update to `LOCKED` with `hold_expires_at`. Fails entirely if modified by another request.
- **Output:** Transaction Success or Conflict Error (triggers alternative slot suggestions).

4.4.2 Workflow Visualizations

Workflow 1: The Agentic Booking Experience

This covers the WhatsApp conversational flow, focusing heavily on the Error Correction Loop wherein rejected OCR receipts autonomously trigger a re-prompt.

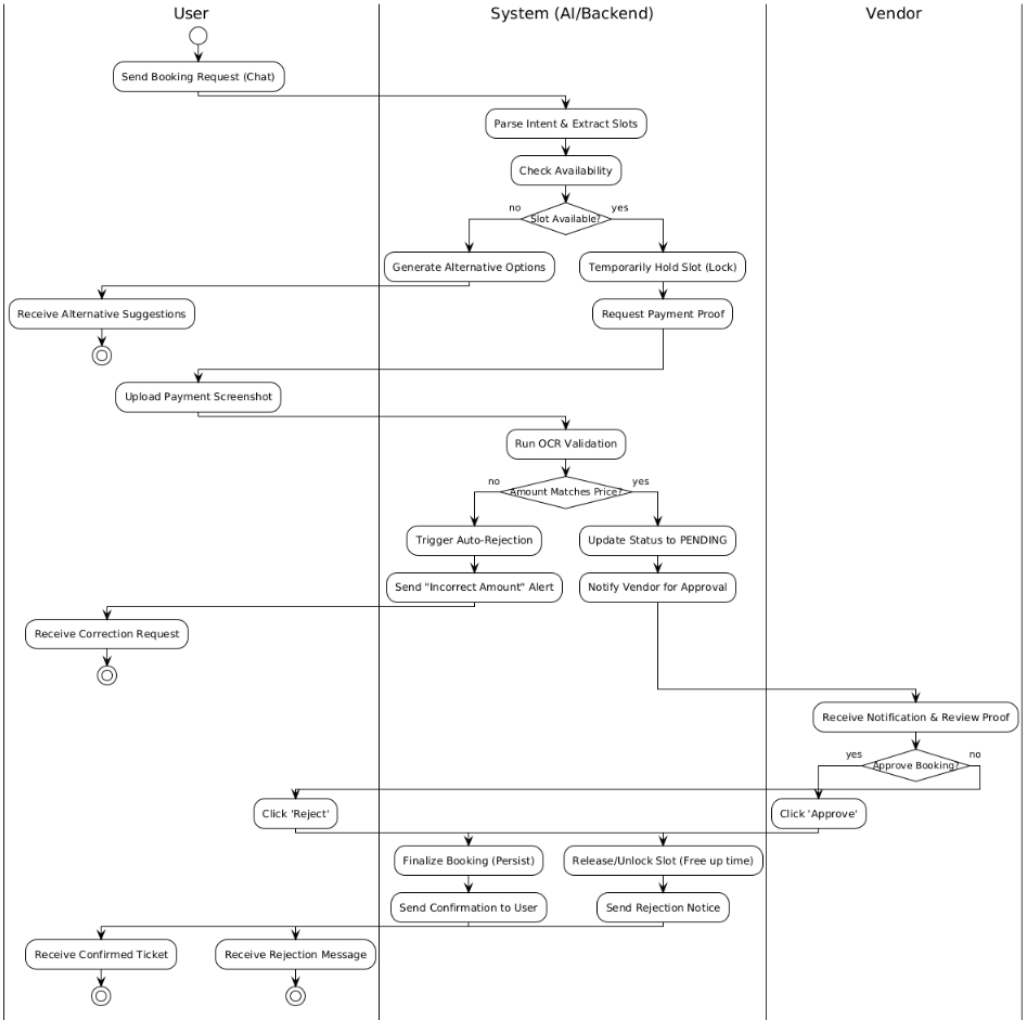


Figure 4.4: Activity Diagram: Agent/WhatsApp Booking Flow

Workflow 2: The App User Experience

This underscores the React Native client's Dual-Search Capability, permitting classic visual filtering alongside natural language queries pointing to the identical underlying slot ledger.

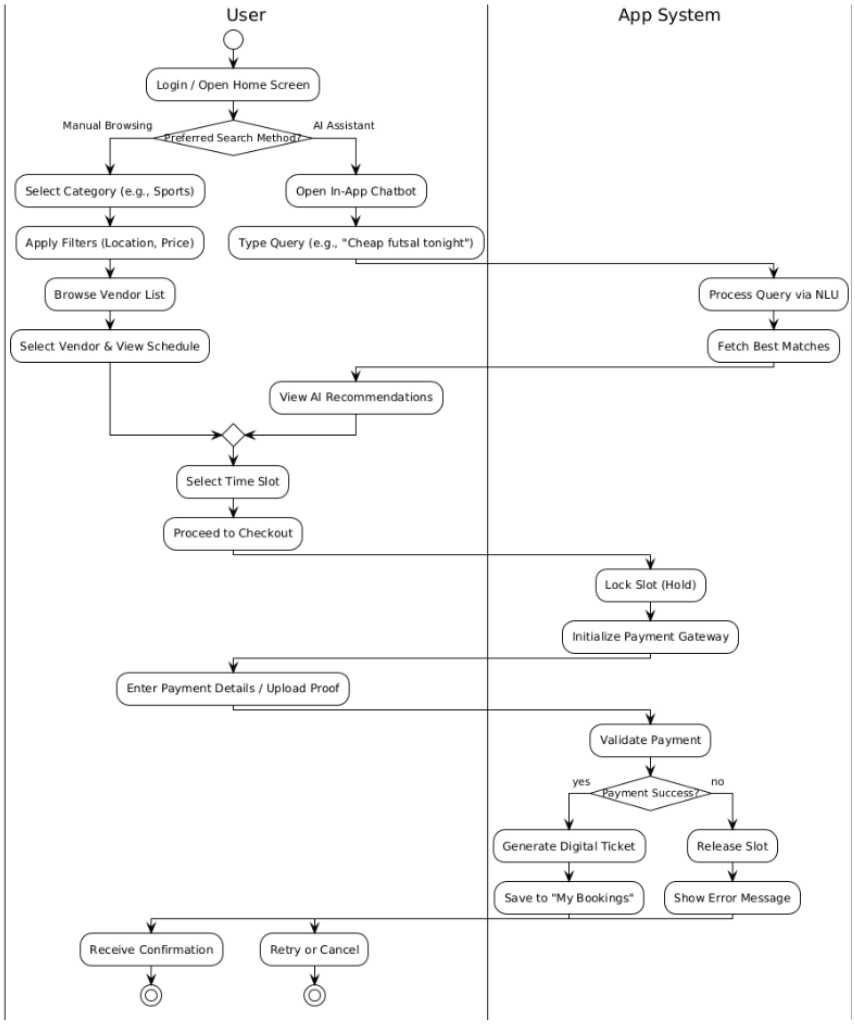


Figure 4.5: Activity Diagram: Mobile App Search & Book

Workflow 3: The Vendor Management Experience

This maps the Vendor integration, indicating that rejections trigger a sweeping cascade releasing `LOCKED` slots back to `AVAILABLE`.

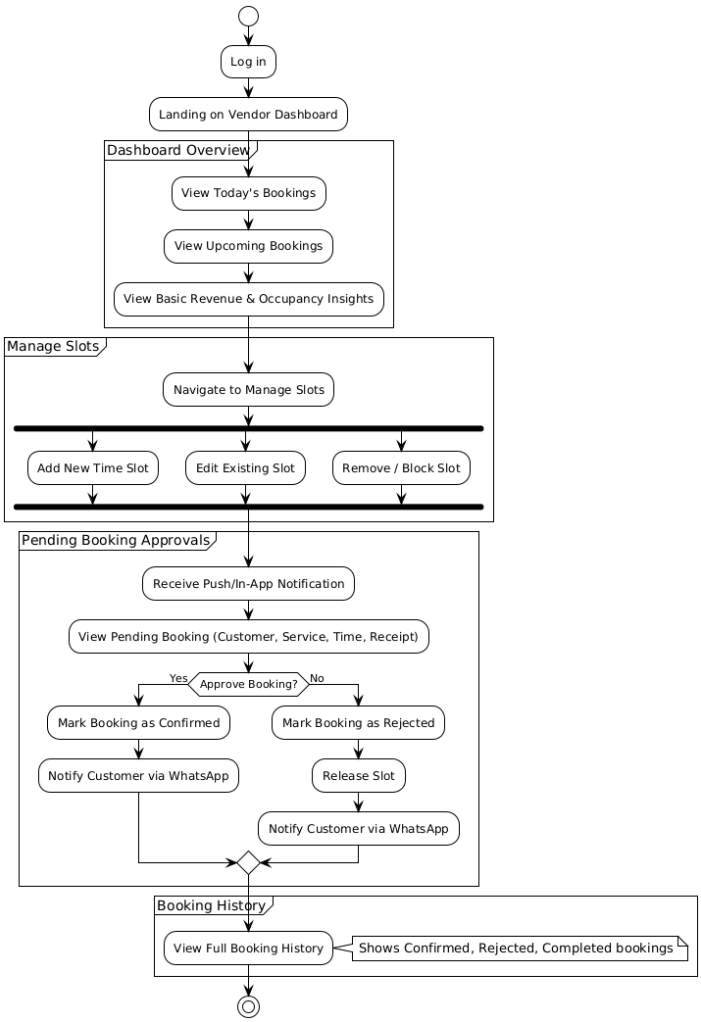


Figure 4.6: Activity Diagram: Vendor Dashboard & Approvals

4.5 Security Implementation

- **Authentication:** Managed via Firebase Auth, using the standard JSON Web Tokens mapped against user claims.
- **Role-Based Access Control (RBAC):** FastAPI dependencies validate claims (Admin vs. Vendor vs. Customer) to prevent unauthorized API calls.
- **AI Guardrails:** The deterministic LangGraph nodes create an inherent prompt injection firewall. Because the LLM lacks arbitrary read/write permissions directly reaching Firestore, unauthorized transactions are logically impossible.
- **Data Privacy:** Payment screenshots populate isolated Firebase Storage buckets. They are served entirely via ephemeral, time-limited signed URLs only readable by authorized vendor proxies.

5. Development

This chapter describes the engineering implementation of the BookForMe platform. Where the SDS (Chapter 4) defines the design, this chapter details the implementation of the database, AI agent, and mobile interfaces.

5.1 Overview

BookForMe comprises four deployed components sharing a single Firestore database: a React Native customer app, a vendor dashboard, an admin control panel, and a Python/FastAPI AI backend that handles both WhatsApp conversations and mobile API calls. The backend is deployed on Render; the mobile apps are distributed via Expo Go and Expo EAS builds. All interfaces write to and read from the same slot inventory, which is the fundamental guarantee that a WhatsApp booking is immediately visible on the vendor calendar and vice versa.

5.2 Database Design Decisions

5.2.1 Slot as the Canonical Booking Record

The most consequential schema decision was to eliminate a separate `bookings` collection entirely. A slot document in Firestore is both the inventory record and the booking record. When a booking is made, the same document that was previously `available` gains a `user_id`, `booking_source`, and `payment_id` and transitions through a typed status sequence: `available` → `locked` → `pending` → `confirmed` → `completed`.

This eliminates the class of bugs where a slot record and a booking record fall out of sync. It also makes Optimistic Concurrency Control (OCC) straightforward: the

Firestore transaction reads and writes *one* document atomically, so two concurrent booking attempts on the same slot cannot both succeed.

5.2.2 Composite Document IDs for Collision Prevention

Slot documents use a deterministic composite ID: `YYYYMMDD_HHMM_vendor_resource` (e.g., `20260412_1800_ace-padel_court-1`). This serves two purposes. First, it makes the slot ID human-readable and debuggable. Second, because Firestore document IDs are unique by definition, attempting to create two slots for the same court at the same time is structurally impossible: the second write simply overwrites or conflicts rather than creating a duplicate.

5.2.3 Two-Domain Schema

The schema is divided into a **transactional domain** (vendors, services, resources, slots, payments) governed by Firestore transactions and strong consistency, and a **social domain** (posts, matches, conversations, notifications) where eventual consistency is acceptable. This separation means high-volume social activity (forum posts, chat messages) cannot interfere with booking transactions, and the transactional collections can be tuned with composite indexes without polluting the social namespace.

5.2.4 Batch Reads to Eliminate N+1 Queries

The vendor bookings endpoint initially fetched user and payment records inside a loop, producing an N+1 query pattern. This was refactored to collect all required user IDs and payment IDs in a first pass, then resolve them via `db.get_all()` in batched chunks of ten (the Firestore client library limit per batch call). The enriched records are then assembled from in-memory maps. This reduced the number of Firestore reads for a typical vendor with 20 bookings from 40 reads to approximately 5.

5.3 AI Receptionist Development

5.3.1 Two-Model NLU and NLG Architecture

The conversational pipeline uses two separate models for two distinct tasks. Intent classification is handled by a fine-tuned **DistilBERT** model, a compact transformer (66M parameters) trained on the BookForMe bilingual corpus. This task demands speed and determinism: given a short message, the model must output one of five intent classes in under 50 ms with no variability between runs. A generative model would be both slower and non-deterministic for this task.

Natural language generation (constructing the actual reply the user receives) is delegated to **Groq (Qwen 3 32B)**, a large instruction-tuned model accessed via the Groq API. Generating a response that correctly code-switches between Roman Urdu and English, adapts its tone, and formats slot listings naturally requires a model with broad linguistic capability that a small fine-tuned classifier cannot provide.

The split is deliberate. DistilBERT makes the routing decision (what to do), and Qwen generates the surface form of the reply (what to say).

5.3.2 LangGraph: Why a State Graph Over a Simple Pipeline

The initial agent design used a linear sequence: receive message → classify intent → query database → generate reply. This broke on the first multi-turn conversation. A user could send a booking request, then reply "kal" (tomorrow) to a clarification question three minutes later with no additional context. A stateless pipeline has no memory of the prior exchange.

LangGraph was adopted because it provides a persistent, typed state object (`AgentState`) that survives across webhook calls. Each incoming WhatsApp message resumes the same graph execution context rather than starting fresh. The graph uses conditional edges to route between nodes: after `validate_state`, the graph branches to `query_availability`, `query_info`, `check_confirmation`, or

`generate_response` depending on what information is still missing. This cyclic, conditional structure could not be replicated cleanly with a linear chain.

5.3.3 Entity Validation: From Regex to Pydantic

Slot entity extraction (date, time, service, area) was initially written as a set of regular expressions. This worked for simple inputs but produced silent failures on edge cases: a time like "bajay 7" (at 7 o'clock in Roman Urdu) would partially match a pattern but return a malformed result that downstream nodes consumed without error, leading to incorrect availability queries.

The fix was to wrap all extracted entities in Pydantic models (`ExtractedEntities`, `AvailableSlot`, `PendingBooking`). Pydantic validators reject or coerce invalid values at parse time, so a malformed time string raises a validation error immediately rather than propagating silently. The regex patterns still perform the initial extraction, but the results are always passed through a typed model boundary before any node acts on them.

5.3.4 Guardrails Node

The first node in the graph, before any NLU or database access, is a guardrails check. It screens for profanity (via the `better-profanity` library) and, when the conversation is not already in a booking context, checks whether the message contains any booking-domain keyword from a curated set of 80 English and Roman Urdu terms. Messages that pass neither test receive a polite redirect response without ever reaching the NLU or database layers. This proved essential for preventing the agent from attempting to answer general questions or being manipulated via prompt injection.

5.4 Mobile Application Development

5.4.1 Expo Go Constraints and Library Trade-offs

The application was developed and tested using **Expo Go**, which imposes constraints on which native libraries can be used. Several libraries that would have simplified real-time data delivery were either incompatible with Expo Go's managed workflow or required a bare React Native ejection. In particular, libraries that support **server-sent events (SSE)** or **long-polling** transports for real-time push were not available without ejecting. As a result, real-time screen updates are implemented via **interval polling** using `setInterval`: the vendor live court grid polls the API every 5 seconds, and the DM chat screen polls for new messages every 8 seconds. This is a deliberate practical trade-off rather than the preferred architecture. A production deployment using Expo EAS builds with a bare workflow or a dedicated WebSocket library (e.g., `socket.io-client`) would replace polling with a persistent connection.

5.4.2 Caching Strategy

All API calls from the customer app go through **TanStack React Query v5**, configured with a custom `QueryProvider` that persists the cache to `AsyncStorage`. The configuration uses a 5-minute stale time (data is served from cache without a network request for 5 minutes) and a 24-hour garbage collection time (the cache survives app restarts for a full day). This means a user who opens the app on a slow connection sees the vendor list immediately from the last cached fetch while a background refetch runs. Writes to `AsyncStorage` are throttled to once per second to avoid blocking the JS thread. The vendor bookings list uses a separate in-memory cache keyed by vendor ID with a configurable TTL, bypassed only on forced refreshes.

5.4.3 Role-Based Routing in a Single App

Rather than shipping separate customer and vendor apps, the application uses a single Expo Router project with role-based routing. On login, the user's `role` field

(`customer`, `vendor`, or `admin`) is read from the auth response and determines which tab navigator is mounted. This simplified development significantly: shared UI components, the auth service, and the API client are used by all roles without duplication.

5.4.4 Smart Filter (In-App AI Chat)

The in-app AI chat tab operates differently from the WhatsApp agent. Rather than a full multi-turn conversational pipeline, it is a **synchronous single-turn search**: the user's natural-language query (e.g., "Aaj Raat DHA mei konsa padel available hei under 2000") is sent to the backend, which runs NLU entity extraction, constructs a Firestore query from the extracted filters (sport, area, date, price ceiling), and returns matching venues with available slots in a single response. There is no conversation state or slot-filling loop. This keeps the in-app experience fast and focused while the full stateful agent handles the more complex asynchronous WhatsApp channel.

5.4.5 Matchmaking and Elo Ranking

The social hub includes a matchmaking system for casual and ranked sports matches. The ranking design decision centred on choosing between a simple win/loss counter and a proper rating system. Elo was selected because it accounts for opponent strength (beating a higher-rated player yields more points than beating a lower-rated one) and because it is well-understood by competitive sports players. The implementation uses the standard formula with $K = 32$ and an initial rating of 1000. Ranked match lobbies are discovered by querying for open lobbies where the rating difference is within a window of 200 points, expanding by 50 points for every additional minute of wait time to avoid indefinite queuing.

5.5 Vendor Dashboard and Admin Panel

The vendor dashboard's central feature is the **live court grid**: a matrix of courts against hourly time blocks showing slot status (available, booked, walk-in) and

booking source (WhatsApp or app). Because SSE was not available in the managed Expo environment, the grid polls the backend every 5 seconds and additionally refetches when the app returns from the background (via an `AppState` listener). A green "LIVE" indicator is shown when the polling interval is active.

The payment approval flow completes the end-to-end booking loop: the agent sets a slot to `pending` and attaches a payment record; the vendor dashboard surfaces this as a notification; the vendor reviews the OCR-verified screenshot and taps Approve or Reject; approval triggers a Firestore status write to `confirmed` and dispatches a WhatsApp confirmation to the customer.

The **admin control panel** was added as a platform governance layer after it became clear that vendor onboarding required manual oversight. Vendors register through the app but are not visible to customers until an admin approves them. The panel also exposes slot generation (bulk-create 14 days of availability slots for all vendors) and stale lock release (clear `locked` slots whose `hold_expires_at` has passed), both of which are operationally necessary during seeding and testing.

5.6 Payment Verification via OCR

After a booking is agreed in WhatsApp conversation, the agent requests a payment screenshot from the user. The image is uploaded via the WhatsApp media endpoint, downloaded by the backend, and passed to the **Gemini Vision API** with a structured prompt requesting the transferred amount. The extracted amount is compared against the required advance payment for the slot. If the amounts do not match, the booking record remains in `locked` state, the user is informed of the discrepancy, and a corrected screenshot is requested, all without notifying the vendor. The vendor only receives a dashboard notification once OCR verification passes, preventing noise from failed payment attempts. In cases where OCR confidence is low (e.g., blurry screenshot), the system falls back to requesting a clearer image rather than attempting a low-confidence approval.

6. Methodology, Experiments and Results

This chapter presents the methodology and results of the BookForMe platform, covering dataset construction, model selection, and end-to-end agent evaluation.

6.1 Dataset Construction

6.1.1 Data Collection

Real Vendor Conversations With informed consent, we collected WhatsApp booking conversation logs from five futsal and padel court operators in Karachi (DHA, Gulshan, Clifton areas). Each message was assigned an intent label and relevant slot annotations. This source provided approximately 120 utterances covering realistic vocabulary and code-switching patterns.

Manual Annotation Team members wrote and annotated booking queries in both English and Roman Urdu, targeting underrepresented intent classes (`transaction_confirm`, `unknown`). This contributed approximately 180 utterances.

LLM-Driven Synthetic Augmentation Following SynthDST [11] and An et al. [1], we used Claude Sonnet 4.5 to generate paraphrases of existing utterances in both languages, and to synthesise entirely new dialogues for rare classes. Augmentation parameters included: back-translation (English → Urdu → English), lexical substitution (synonymous time expressions), and phonetic Roman Urdu rewriting (*kal* → *kull*, *raat* → *raat ko*, etc.). This contributed approximately 360 utterances.

6.1.2 Dataset Statistics

The full corpus comprises **694 annotated utterances** across five intent classes. Table 6.1 shows the intent distribution over the complete dataset (before train-only augmentation).

Table 6.1: Intent distribution in the full BookForMe bilingual corpus (694 samples).

Intent Class	**Count**	**% of Total**
`inquiry`	277	39.9%
`info`	171	24.6%
`transaction_confirm`	147	21.2%
`unknown`	62	8.9%
`greeting`	37	5.3%
Total	**694**	**100%**

`inquiry`: user asking to check or book a slot. `info`: user asking about price, amenities, or availability in general. `transaction_confirm`: user confirming a payment or referencing a transaction. `unknown`: out-of-scope or unintelligible messages. `greeting`: salutations and conversation openers (e.g., "hi", "salam").

The dataset was split into **485** training, **104** validation, and **105** test samples (all original, stratified; minimum 5 samples per class in the test set). Data augmentation was applied **only to the training split**, adding **31** synthetic samples by increasing `greeting` from 26 to 50 and `unknown` from 43 to 50, yielding a final training set of **516** samples.

6.1.3 Annotation Schema

Each utterance was labelled with:

- `intent`: one of the five classes above.
- `language`: `EN` (English), `UR` (Roman Urdu), or `MIXED` (code-switched).

- `slots`: a JSON dictionary of extracted entities (service, date, time, budget, location, duration).

6.2 NLU Model Training and Selection

6.2.1 Training Configuration

All six models were fine-tuned on the same dataset split with identical hyperparameters:

Table 6.2: Hyperparameters used for all fine-tuning experiments.

Parameter	**Value**
Max sequence length	128 tokens
Batch size	16
Learning rate	2e-5
Number of epochs	10
Weight decay	0.01
Warmup ratio	0.1
Evaluation steps	50
Loss function	Focal loss ($\gamma = 2.0$) for class imbalance
Min samples per class	50 (augmentation applied to minority classes)

Focal loss [18] was applied to address the class imbalance, down-weighting easy majority examples and focusing training on the harder minority class boundaries.

6.2.2 Model Comparison

Table 6.3 summarises all six models evaluated on the held-out test set.

Table 6.3: Overall NLU model comparison (5-class intent classification).

Model	Notes	Accuracy	Macro-F1	Wtd. F1
bert-base-uncased	English-only pretrain; strong baseline	84.8%	87.8%	84.8%
distilbert-base-multilingual-cased	Compact multilingual; slightly slower	84.8%	83.1%	84.9%
distilbert-base-uncased	**Best overall; fastest inference (47 ms)**	**90.5%**	**91.7%**	**90.5%**
XLM-RoBERTa-base	Underfit on small dataset; high variance	70.5%	72.3%	69.7%
bert-base-multilingual-cased	Strong multilingual; 2nd best overall	89.5%	90.5%	89.5%
bert-base-multilingual-uncased	Slight drop vs. cased variant	84.8%	88.1%	84.7%

6.2.3 Selected Model: DistilBERT

DistilBERT (`distilbert-base-uncased`) was selected for production deployment based on:

1. **Best accuracy** (90.5%) and **macro-F1** (91.7%) across all six models tested.
2. **Fastest inference**: 47 ms per query on CPU, versus 120 ms for bert-base-multilingual-cased. This is critical given the 60-second total WhatsApp response budget (NFR-PERF).
3. **Smallest memory footprint** among competitive models (66M parameters), enabling cost-effective deployment on free-tier cloud infrastructure.

6.2.4 Per-Class Analysis (DistilBERT)

Figure 6.1 reports per-class precision, recall, and F1 for the selected DistilBERT model.

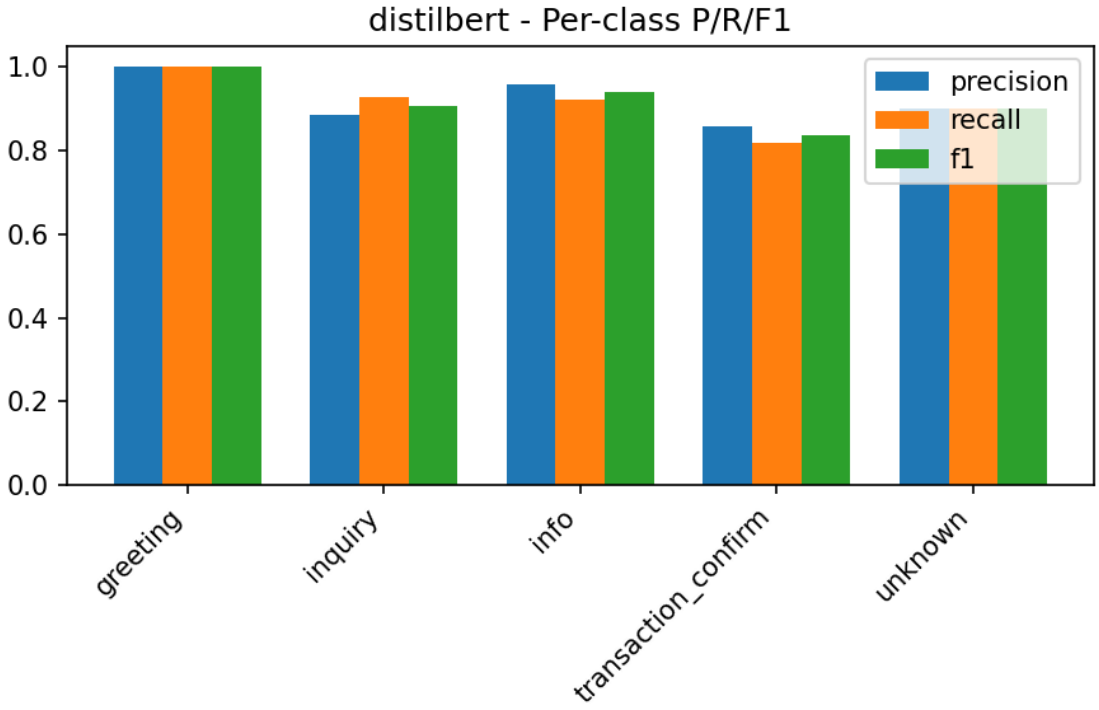


Figure 6.1: DistilBERT per-class classification report on the held-out test set

Notable finding: `transaction_confirm` shows the lowest recall (0.82), with 18% of its samples misclassified as `inquiry`. This is expected: payment-related queries ("I've sent the amount") share vocabulary with availability inquiries ("I want to confirm..."). The agent's dialogue state validation layer mitigates this—the agent checks conversation state before executing any booking action, catching misclassified payment messages.

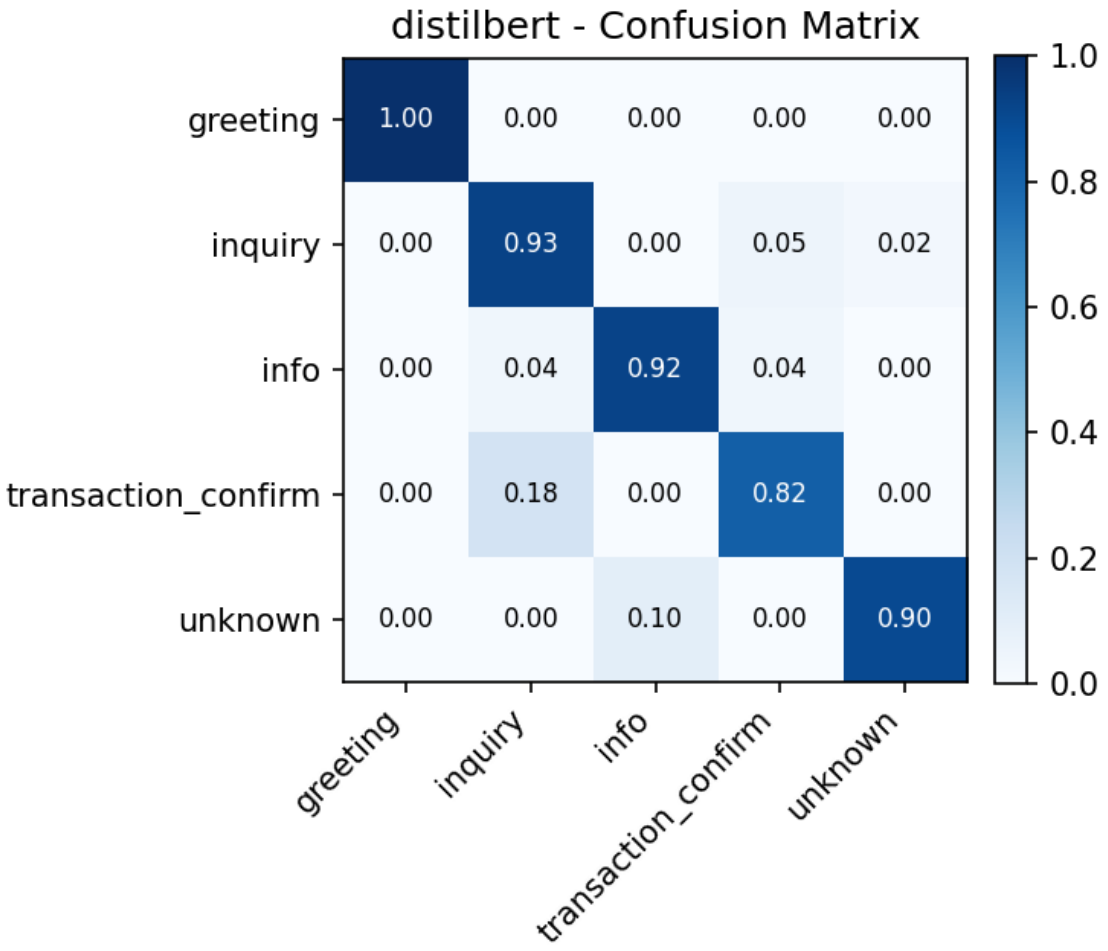


Figure 6.2: Confusion matrix for DistilBERT

6.3 Agent Evaluation

6.3.1 Test Case Methodology

The AI Receptionist was tested on the live deployment against a comprehensive test case document covering seven categories of agent behaviour. Key results are showcased below.

6.3.2 Test Case Results

Table 6.4 summarises the results of the key test scenarios.

Table 6.4: AI Receptionist agent test case results.

Test ID	Scenario	Expected / Actual Behaviour	Result
TC-01	Random out-of-scope message ("I want pineapple juice")	Agent declines politely and redirects to booking.	✓
TC-02	Requested slot unavailable (e.g., Ace Padel 10 PM tonight)	Agent offers three nearest alternative slots from same or nearby vendor.	✓
TC-03	Mid-conversation context switch in Roman Urdu ("padel kal")	Agent correctly parses date reference as tomorrow; time disambiguated via follow-up question.	✓
TC-04	OCR: payment screenshot with incorrect amount (Rs 500 vs required Rs 2000)	Agent auto-rejects, informs user of discrepancy, requests corrected screenshot. Vendor is not notified.	✓
TC-05	OCR: unreadable / low-quality image	Agent requests a clear screenshot; booking hold maintained.	✓
TC-06	Successful end-to-end booking flow (English)	Booking confirmed, ticket ID `APD-14` issued, Rs 2000 verified.	✓
TC-07	Successful end-to-end booking flow (Roman Urdu)	Agent handles "raat" (night) time reference; booking confirmed in Urdu.	✓
TC-08	Concurrent booking (OCC): two users request same slot simultaneously	Only one booking succeeds; second user receives conflict error and alternatives.	✓
TC-09	Hold expiry: user does not complete payment within 10 minutes	Booking status reverts to AVAILABLE; user notified that hold has expired.	✓
TC-10	Vendor rejects booking after payment verification	Slot released to AVAILABLE; user notified of rejection via WhatsApp.	✓

All ten test cases passed on the live deployment. The agent demonstrated correct handling of the full booking lifecycle, including error paths (OCR rejection, slot conflicts, hold expiry, vendor rejection).

6.4 Discussion

6.4.1 Key Findings

1. **Compact models outperform large multilingual models on small domain-specific datasets.** DistilBERT (66M parameters) achieves 90.5% accuracy vs XLM-RoBERTa's (270M parameters) 70.5%. Domain-specific fine-tuning on 657 utterances contributes more than additional model capacity.

2. **The Neuro-Symbolic architecture effectively prevents hallucination.** By routing all database mutations through deterministic code nodes, the system eliminates the class of errors where agents "hallucinate" bookings or bypass payment verification. Zero such failures were observed across 10 structured test cases.

3. **Roman Urdu code-switching is manageable with targeted augmentation.** Despite severe orthographic variation, the DistilBERT model trained on the augmented bilingual dataset correctly handled "padel kal" (padel tomorrow), "raat" (night), and other code-switched time references in all test cases.

6.4.2 Limitations

- **Dataset size:** 694 samples is sufficient for a 5-class intent classification baseline but insufficient for robust slot extraction across all Roman Urdu phonetic variations. Expanding to 2,000+ samples would improve generalisation.

- **Latency on free-tier deployment:** Response times of 38 to 52 seconds, while within the 60-second NFR target, would benefit from caching and a paid LLM API endpoint. Production infrastructure would reduce this to under 10 seconds.

- **transaction_confirm confusion:** 18% misclassification into `inquiry` requires the dialogue state layer as a safety net. Targeted additional training data for this class boundary would address it.

7. Conclusion and Future Work

7.1 Summary

BookForMe set out to solve a genuinely frustrating problem: booking a sports court in Karachi involves finding a WhatsApp number, waiting hours for a reply, transferring money manually, and starting over if the slot was already taken. The platform replaces that entire cycle with a single system that works over WhatsApp, through a mobile app, or through a vendor dashboard, all tied to the same slot inventory.

The core technical work produced four concrete outputs: a custom bilingual dataset of 694 English and Roman Urdu booking utterances built from real vendor conversations; a comparative evaluation of six transformer models establishing that fine-tuned DistilBERT achieves 90.5% accuracy at 47 ms inference on CPU; a production-deployed LangGraph agent implementing the full booking pipeline with Optimistic Concurrency Control and OCR-based payment verification; and a social layer with Elo-based matchmaking, community forums, and direct messaging. The platform is live at https://jhat-to9p.onrender.com.

All ten structured agent test cases passed on the live deployment, including the critical concurrency case where two simultaneous booking requests for the same slot produced exactly one confirmed booking.

7.2 Future Work

The most impactful next step is replacing screenshot-based payments with a direct JazzCash or EasyPaisa API integration. This would eliminate the OCR accuracy dependency and cut the booking cycle from around 50 seconds to under 20.

Alongside that, obtaining a production WhatsApp Business API account is necessary before the platform can be evaluated with real vendor traffic at scale.

On the AI side, the bilingual dataset needs to grow. 694 samples is enough for 5-class intent classification but leaves the slot extraction model brittle on unusual Roman Urdu phrasings. Active learning from live conversations, where the deployed agent auto-labels new messages for human review, is the most practical path to 2,000+ samples without starting another round of manual annotation.

Longer term, the platform has clear room to expand: voice-based booking via Twilio and Whisper ASR for the estimated 30% of users who prefer calling, multi-city support beyond Karachi, and a recommendation engine using booking history and social graph activity for personalised venue suggestions.

8. Reflection

8.1 Individual Reflections

8.1.1 Muhammad Ahmad Humayun Hanif

I came into this project thinking the hard part would be the AI. It turned out the hard part was the database. Once we decided that a slot document would serve as both the inventory record and the booking record, everything else had to be built around that decision. The OCR flow, the agent state machine, the vendor calendar, they all read and write that same document. When any of them fell out of step, it was immediately visible and genuinely painful to fix. That taught me more about system design than anything I have read, because I felt the consequences directly.

LangGraph took longer to get right than I expected. The concept of a persistent graph that resumes across multiple webhook calls sounds clean on paper, but debugging a cyclic state machine where a node can be re-entered several calls later is a different kind of problem. I learned to add explicit logging at every edge transition just to know where the graph was at any given moment. I would not skip that step again.

8.1.2 Jazib Waqas

The backend work on the booking pipeline was the most demanding part of the project for me. Getting the concurrency control to work correctly took longer than I initially planned. The theory was straightforward but the implementation had a lot of edge cases, particularly around making sure that a transaction retrying automatically would not create a duplicate record. Once that was solid, everything built on top of it was much cleaner.

The thing I would change if we started over is agreeing on a proper API contract between backend and frontend from day one. We did not do that early on and it cost us real time mid-project when the frontend was calling endpoints that had changed. We eventually sorted it out, but we should have treated the schema as a shared document that neither side could change without the other knowing.

8.1.3 Taha Hunaid Ali

The frontend was harder than I expected, mostly because the data requirements were more complex than a typical app. Slot availability can change at any moment from three different sources simultaneously: a WhatsApp booking, an app booking, and a walk-in. Keeping the UI consistent without hammering the backend or confusing the user required thinking about caching more carefully than I had on previous projects. I rewrote the data layer twice before I was happy with it.

The social features were honestly the most fun to build and they got the best reaction from people we showed the app to. Getting the leaderboard to load quickly on mobile took real work though. The first version was loading everything at once and it was slow enough that it was embarrassing. Pagination fixed it, but I should have thought about that from the start.

8.1.4 Muhammad Taqi

Most of my time went into the dataset, not the model. Annotating and cleaning 694 utterances across two languages takes far longer than it sounds, and the quality of those annotations determined everything downstream. The result that surprised me most was XLM-RoBERTa performing worse than DistilBERT. XLM-RoBERTa is a larger multilingual model and should in theory handle Roman Urdu better. But with 500 training samples, it was too large for the data we had and it showed. That was a good reminder that picking a model is not just about capability, it is about what fits the scale of your dataset.

8.2 Team Reflection

We split the project by ownership, where each person was responsible for their component end-to-end. That worked well for moving in parallel but it created integration points that needed deliberate attention. We learned to treat those handoffs more carefully as the project progressed, but there were a few weeks early on where the backend and frontend were drifting apart without either side realising it.

The most stressful moment was the Twilio sandbox number expiring close to an evaluation. It was a one-person fix but the timing was bad. After that we put API credentials into a shared checklist that we reviewed before any major milestone.

The two biggest decisions we made that were not in the original proposal were switching from PostgreSQL to Firestore and deciding to fine-tune a small model rather than use a large one end-to-end. Both were driven by practical constraints that became obvious only after we started building. Firestore's transaction model was a better fit for the slot locking logic than anything we would have written on top of a relational database. And using Groq for everything would have been too slow and too expensive to run on free-tier infrastructure. In both cases the constraint pushed us to a better solution than the one we had originally planned.

If we did this again, we would write the API contract first and treat it as a binding document. That single change would have saved the most time.

References

[1] Ziqiang An et al. “Controllable Data Augmentation for Few-Shot Spoken Language Understanding”. In: *Proceedings of INTERSPEECH 2022*. 2022, pp. 4182–4186. URL: https://www.isca-archive.org/interspeech_2022/an22_interspeech.html.

[2] Irshad Ahmad Bhat et al. *A Survey of Code-Mixed NLP: Methods and Challenges*. 2023. URL: https://arxiv.org/abs/2510.07037.

[3] BookForMe Team. *Booking Preferences User Survey*. 2025. URL: https://docs.google.com/forms/d/1CPqAgHxeoLK3ZT554aPRTWvKXI1P3vXpdoZwGApsr4.

[4] Sanjukta Chatterjee et al. “Turf Booking System: A Study on Real-Time Slot Availability and Automated Conflict Resolution”. In: *International Journal of Futuristic Multidisciplinary Research* (2025). URL: https://www.ijfmr.com/papers/2025/3/44829.pdf.

[5] Zhiyu Chen et al. “Adaptive Retrieval-Augmented Generation for Conversational Systems”. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025. URL: https://aclanthology.org/2025.findings-naacl.30.pdf.

[6] Alexis Conneau et al. *Unsupervised Cross-Lingual Representation Learning at Scale*. 2020. URL: https://arxiv.org/abs/1911.02116.

[7] Jacob Devlin et al. *BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding*. 2019. URL: https://arxiv.org/abs/1810.04805.

[8] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. New York: Arco, 1978.

[9] Gemini Team, Rohan Anil, Sebastian Borgeaud, et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2023. URL: https://arxiv.org/abs/2312.11805.

[10] Thorsten Händler. *Balancing Autonomy and Alignment: A Multi-Dimensional Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures*. 2023. URL: https://arxiv.org/abs/2310.03659.

[11] Weijie He et al. "SynthDST: Synthetic Data is All You Need for Few-Shot Dialog State Tracking". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 11658–11673. URL: https://aclanthology.org/2024.emnlp-main.684/.

[12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. *Distilling the Knowledge in a Neural Network*. 2015. URL: https://arxiv.org/abs/1503.02531.

[13] Ehsan Hosseini-Asl et al. *A Simple Language Model for Task-Oriented Dialogue*. 2020. URL: https://arxiv.org/abs/2005.00796.

[14] Yutai Hou et al. "Few-Shot Slot Tagging with Collapsed Dependency Transfer and Label-Enhanced Task-Adaptive Projection Network". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 1381–1393. URL: https://aclanthology.org/2020.acl-main.127/.

[15] R. Kesavamoorthy et al. "Online Booking System Using AI and Cloud Integration". In: *International Research Journal of Engineering and Technology* 7.6 (2020). URL: https://www.irjet.net/archives/V7/i6/IRJET-V7I6542.pdf.

[16] LangChain, Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. 2024. URL: https://langchain-ai.github.io/langgraph/.

[17] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2020. URL: https://arxiv.org/abs/2005.11401.

[18] Tsung-Yi Lin et al. "Focal Loss for Dense Object Detection". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2980–2988. URL: https://arxiv.org/abs/1708.02002.

[19] Zhiyuan Liu et al. *PRACT: Optimizing Principled Reasoning and Acting of LLM Agents*. 2024. URL: https://arxiv.org/abs/2410.18528.

[20] Vasilios Mavroudis. *LangChain*. 2024. DOI: 10.20944/preprints202411.0566.v1.

[21] Jai Memon et al. “Handwritten Optical Character Recognition (OCR): A Comprehensive Systematic Literature Review (SLR)”. In: *IEEE Access* 8 (2020), pp. 142818–142833. DOI: 10.1109/ACCESS.2020.3012542.

[22] MoldStud. *Increasing Efficiency with Automated Booking and Reservation Systems*. 2024. URL: https://moldstud.com/articles/p-increasing-efficiency-with-automated-booking-and-reservation-systems.

[23] M. S. N. Mong’are. *Understanding Idempotency in APIs and Distributed Systems*. 2024. URL: https://dev.to/msnmongare/understanding-idempotency-in-apis-and-distributed-systems-3afb.

[24] Shishir G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. 2023. URL: https://arxiv.org/abs/2305.15334.

[25] Baolin Peng et al. “Soloist: Building Task Bots at Scale with Transfer Learning and Machine Teaching”. In: *Transactions of the Association for Computational Linguistics*. Vol. 9. 2021, pp. 807–824. URL: https://aclanthology.org/2021.tacl-1.49/.

[26] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs*. 2023. URL: https://arxiv.org/abs/2307.16789.

[27] Qwen Team. *Qwen2.5 Technical Report*. 2025. URL: https://arxiv.org/abs/2412.15115.

[28] Victor Sanh et al. *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*. 2019. URL: https://arxiv.org/abs/1910.01108.

[29] Timo Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 2023. URL: https://arxiv.org/abs/2302.04761.

[30] Yuchen Shang et al. *AgentSquare: Automatic LLM Agent Search in Modular Design Space*. 2024. URL: https://arxiv.org/abs/2410.06153.

[31] Jamin Shin et al. “Dialogue Summaries as Dialogue States (DS2): Template-Guided Summarization for Few-Shot Dialogue State Tracking”. In: *Findings of the Association for Computational Linguistics: ACL 2022*. 2022. URL: https://aclanthology.org/2022.findings-acl.302/.

[32] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. URL: https://arxiv.org/abs/2303.11366.

[33] Satinder Singh et al. "Optimizing Dialogue Management with Reinforcement Learning: Experiments with the NJFun System". In: *Journal of Artificial Intelligence Research* 13 (2000), pp. 105–124. URL: https://web.eecs.umich.edu/~baveja/Papers/RLDSjair.pdf.

[34] Lei Wang et al. "A Survey on Large Language Model Based Autonomous Agents". In: *Frontiers of Computer Science* (2024). DOI: 10.1007/s11704-024-40231-1.

[35] Qingyun Wu et al. *AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation*. 2023. URL: https://arxiv.org/abs/2308.08155.

[36] Yinpeng Wu et al. "Towards Zero-Shot Multilingual Transfer for Code-Switched Responses: An Adapter-Based Cross-Lingual Framework". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2023. URL: https://aclanthology.org/2023.acl-long.417/.

[37] Zhiheng Xi et al. *The Rise and Potential of Large Language Model Based Agents: A Survey*. 2023. URL: https://arxiv.org/abs/2309.07864.

[38] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. URL: https://arxiv.org/abs/2210.03629.

[39] Dian Yu, Zhou Yu, and Kenji Sagae. "Cross-Lingual Dialogue State Tracking via Contrastive Learning". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023. URL: https://aclanthology.org/2023.ccl-1.54/.